

Computer Graphics System Outlines

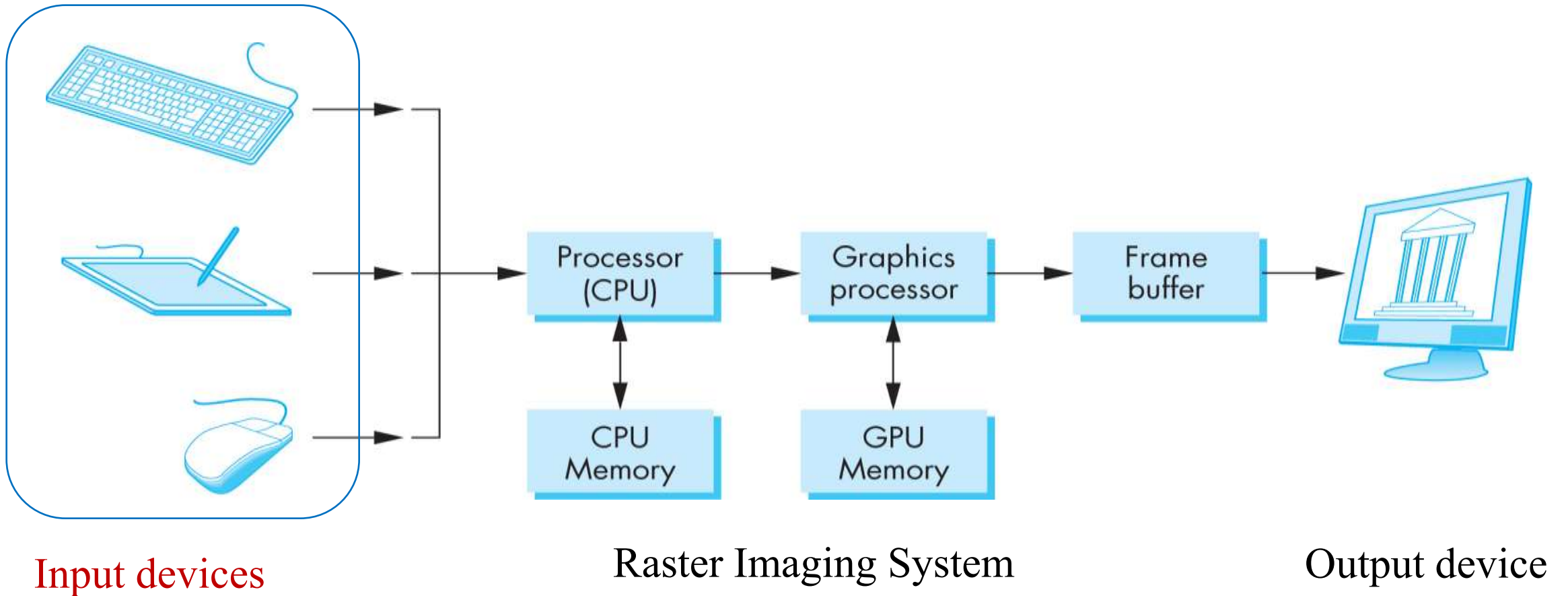
Hardware

- Input devices-输入设备
- Output device-输出设备
- Process device* -显卡

Software

- Image Formation Principle –成像原理
- Rendering Pipeline –渲染管线
- The Programmer's Interface-应用编程接口CG API

Basic Graphics System



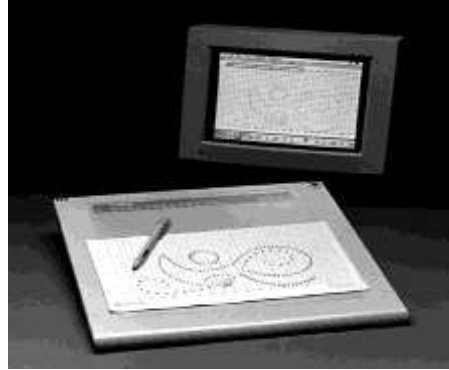
Input Devices

➤ Keyboard

➤ Mouse

➤ Tablet/stylus

- consist of a flat drawing board and a pen
- electromagnetic
- high resolution



➤ Joystick

the rate of cursor movement
depends on the displacement
of the stick

➤ 3D handheld laser scanning digitizer

➤ Touch-screen monitors

➤ Motion Tracking sensors



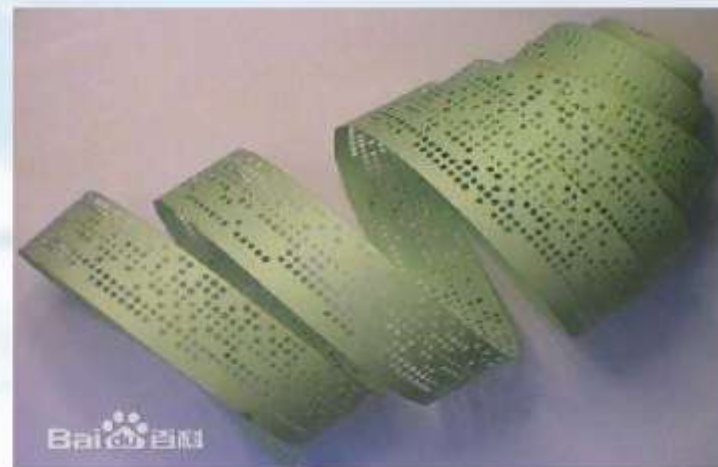
Input Devices (cont.)

- Ref: 大学MOOC 万琳: 计算机图形学

1

输入设备的四个发展阶段

第一阶段：由设计者本人利用**控制开关**、**穿孔纸**等，采用手工操作去适应现在看起来十分笨拙的计算机



Input Devices(cont.)

1

输入设备的四个发展阶段

第二阶段：计算机的主要使用者——程序员利用键盘、光笔等输入设备，采用批处理作业语言或交互命令语言的方式和计算机打交道



Input Devices(cont.)

1

输入设备的四个发展阶段

第三阶段：出现了图形用户界面和各种交互设备，**不懂计算机的普通用户**也可以熟练使用

用户的变化：

设计者 —————> 程序员 —————> 普通用户



Input Devices(cont.)

1 输入设备的四个发展阶段

第四阶段：多通道、多媒体的智能化人机交互阶段

趋势一：以便携电脑、智能手机为代表的计算机的微型化、随身化、嵌入化



1 输入设备的四个发展阶段

第四阶段：多通道、多媒体的智能化人机交互阶段

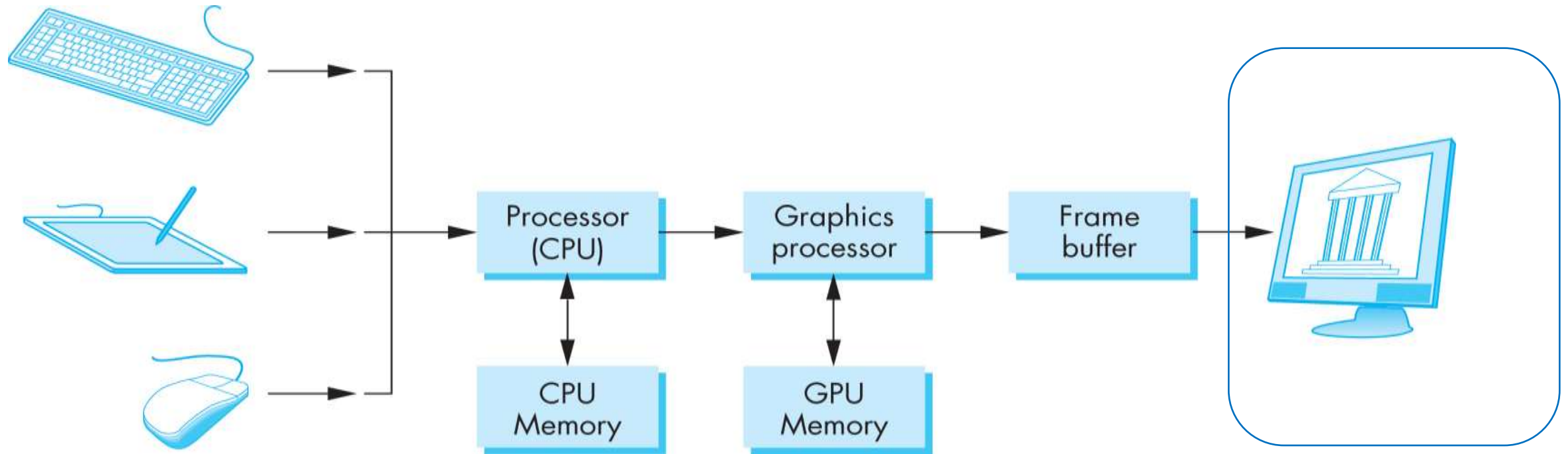
趋势二：以虚拟现实为代表的计算机系统的拟人化



Computer Graphics System Outlines

- Input devices
 - Output device
 - Process device*
-
- Image Formation Principle –成像原理
 - Rendering Pipeline –渲染管线实现*
 - The Programmer's Interface-程序员编程接口

Basic Graphics System



Input devices

Raster Imaging System

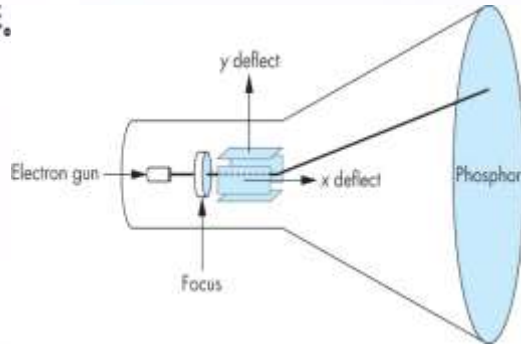
Output device

Display device

- 阴极射线管显示器

阴极射线管

CRT (Cathode Ray Tube) 是一种真空器件，它利用电磁场将高速的、经过聚焦的电子束，偏转到屏幕的不同位置轰击屏幕表面的荧光材料而产生可见图形。



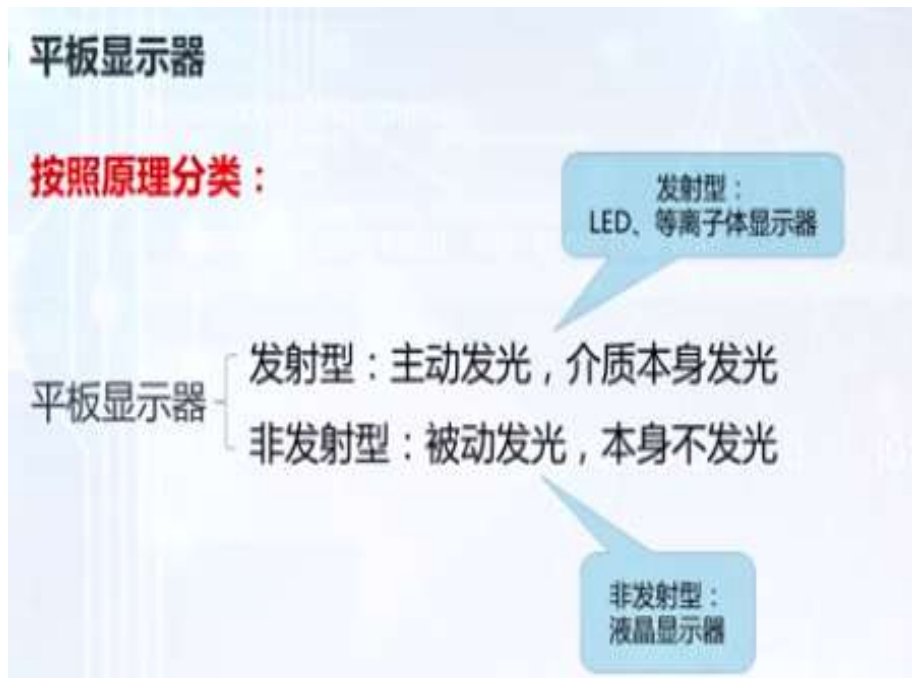
阴极射线管

从外形上看，CRT分为：管颈部分、锥体部分、屏幕部分



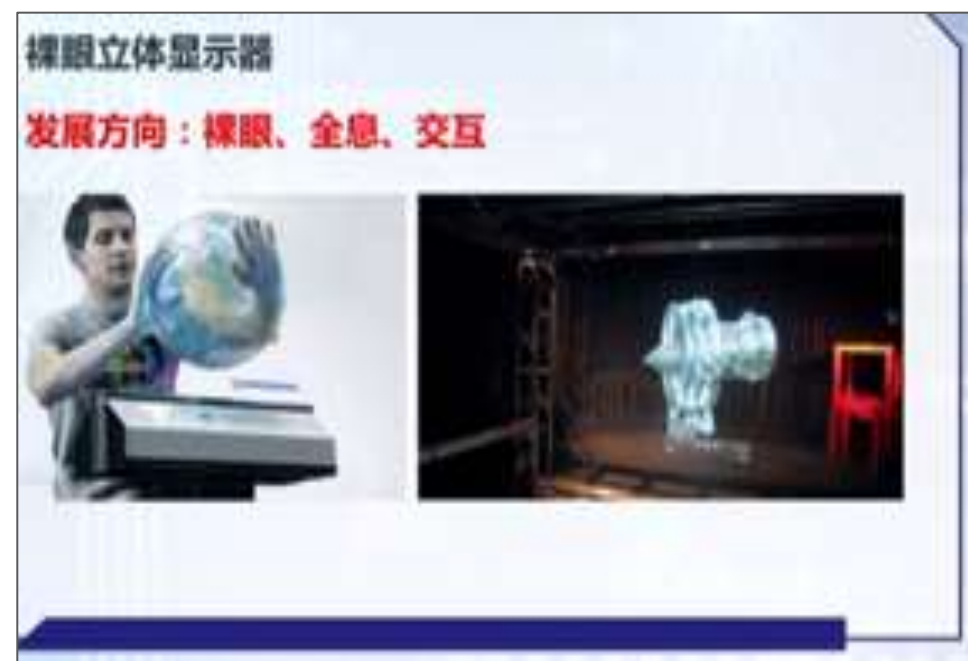
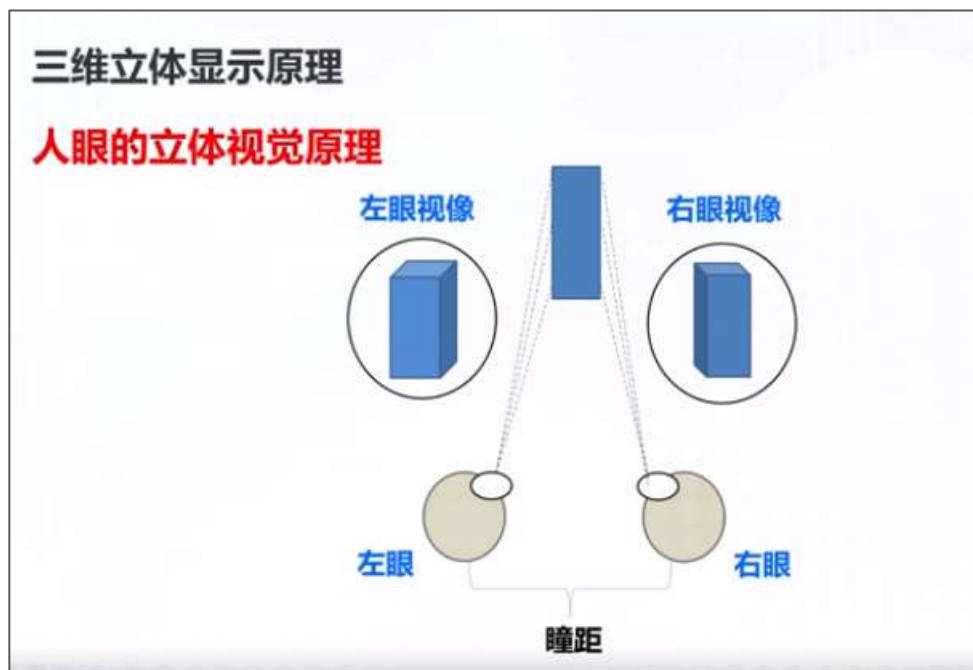
Display device(cont.)

• 平板显示器



Display device(cont.)

- 立体显示器



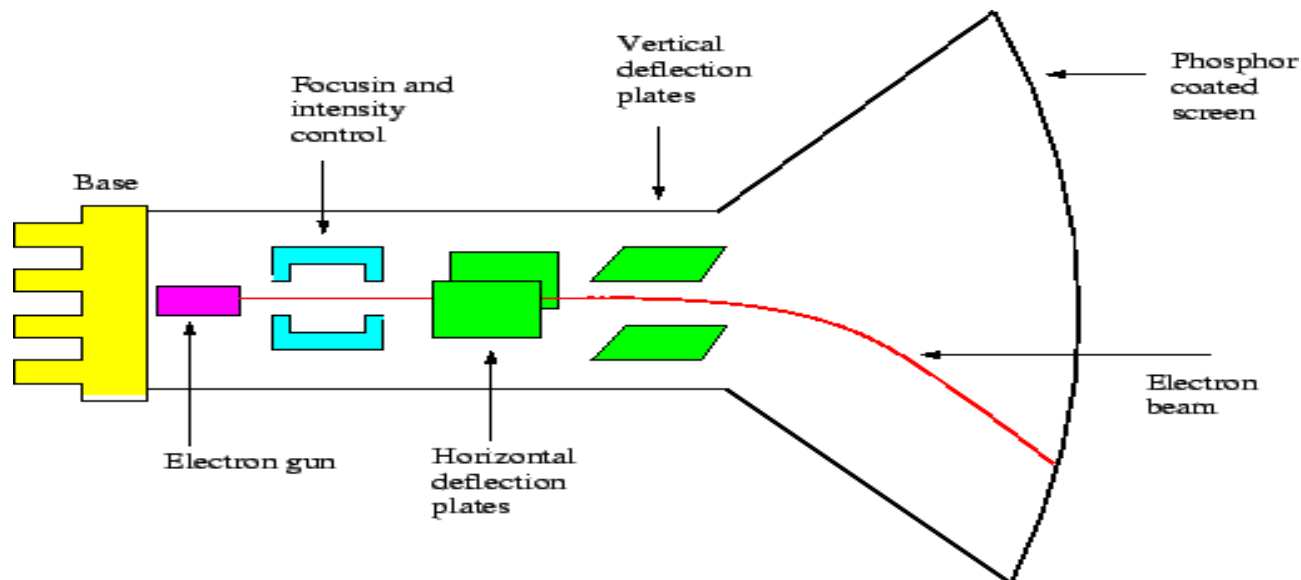
Display device(cont.)



- CRT(Cathode-ray tube display 阴极射线管):[演示](#)

➤ CRT Components

electron gun电子枪; focus聚焦; x&y deflect偏转; phosphor荧光体



对电子束的要求：

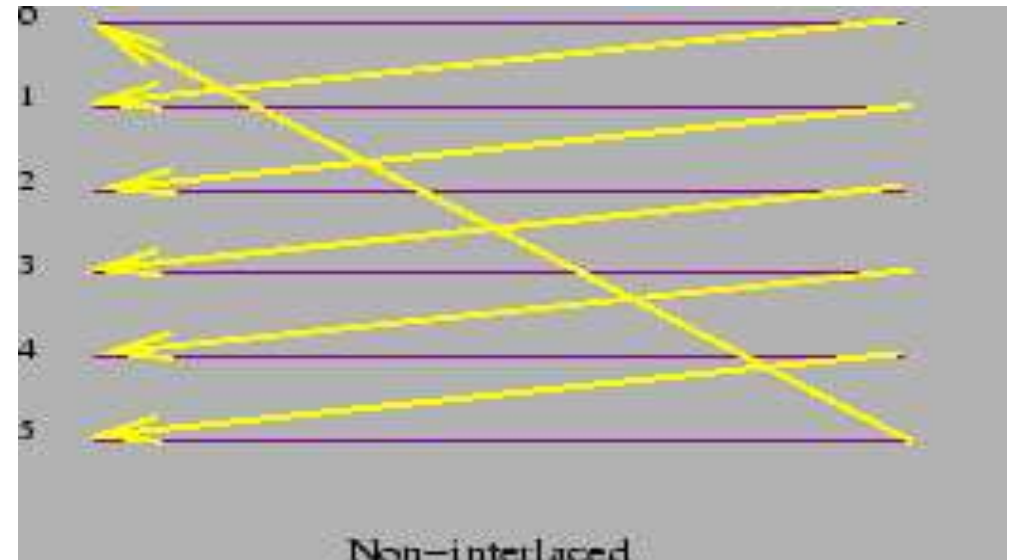
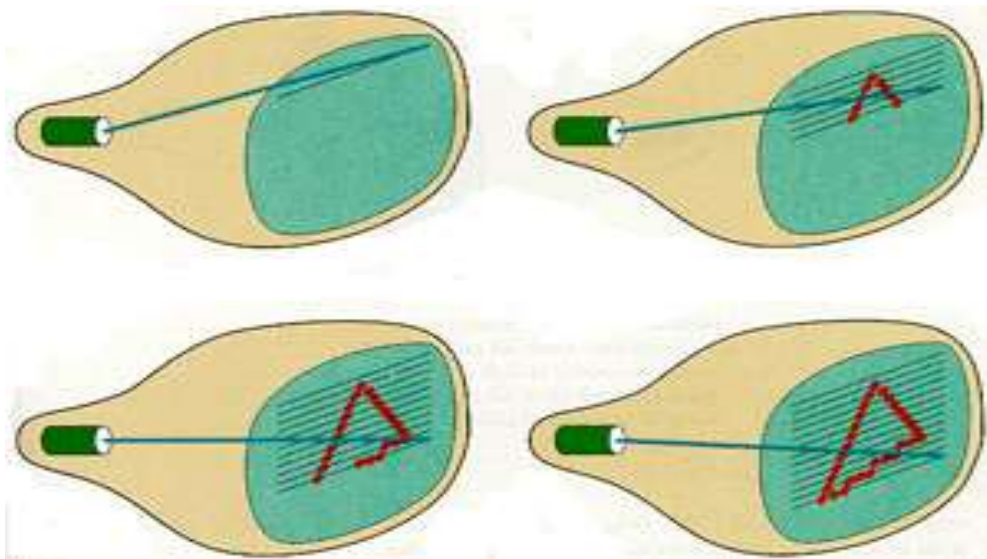
- 强度足够大
- 大小、有无应该可控
- 充分聚焦

Display device(cont.)

CRT(Cathode-ray tube display 阴极射线管)(cont.)

➤ 光栅扫描: The beam 光线 moves regularly in the trails (路径) [演示](#)

The screen of a computer monitor consists of lines numbered 0, 1, 2, ... starting from the top, Each lines consists of luminous spot numbered from left to right



Display device(cont.)

□ **Pixel像素**: Picture element.

□ **Resolution分辨率**: The number of pixels in the framebuffer

□ **Bitmap/Pixelmap** 是一种由像素构成的图像，每个像素都有其特定的颜色和位置。它的核心特征是**分辨率**和**色彩深度**。常见的像 JPG, PNG, GIF, BMP 等格式的文件都是位图/像素图。它的最大缺点是放大后会失真变模糊。



Resolution: 17 * 13



Resolution: 51 * 26



Resolution: 102 * 52

Display device(cont.)

- CRT(Cathode-ray tube display 阴极射线管)(cont.)

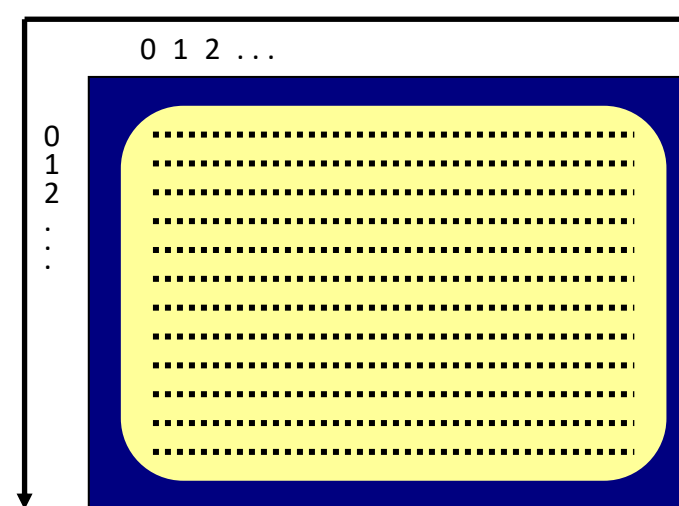
- 光栅扫描显示器基本概念

- **Light spot (光点)** : 最小像素
- **Raster(光栅)** : A bitmap image, consisting of a grid of pixels 像素网格

- **physical resolution (物理分辨率)**

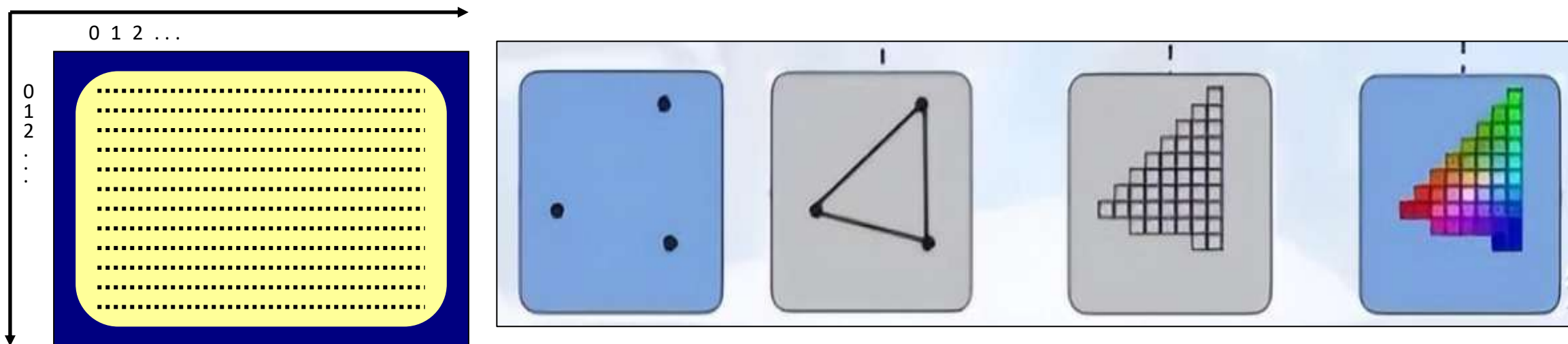
- 屏幕上的二维阵列所包含的光点数
- 显示器的最大显示分辨率
- Example:Dimensions of the screen
 - SVGA(800 *600)
 - XVGA(1024*768)
 - XVGA(1280*1024)

- **display resolution(显示分辨率)**



Question

- 1.显示器为什么需要光栅化Rasterization?



回答：计算机屏幕是数字化的，它只能显示**离散的**像素点，无法真正显示连续的、数学上完美的“线”或“面”。

因此，必须有一个过程来将理想的几何图元“适配”到像素网格上，这个过程就是光栅化。

光栅化是现代实时图形渲染（如游戏、交互式应用）的基石。

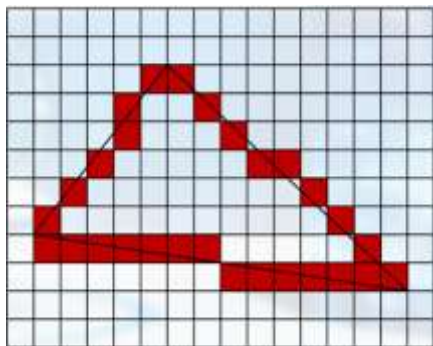
Display device(cont.)

- CRT(Cathode-ray tube display 阴极射线管)(cont.)

- When the beam hits the phosphor(荧光体) on the screen, the phosphor (荧光粉) lights up (亮起来), but its intensity(亮度) decays(衰减) very fast.
- Nonetheless, the image retains in the retinas(视网膜) of our eyes for about 1/20 sec.

➤ **Question: 如何保证CRT光栅显示器的屏幕不闪烁?**

- The entire screen is refreshed 30 to 60 times per second so that we can see a steady picture (整个屏幕每秒刷新30到60次, 这样我们就可以看到一个稳定的画面)
- Refresh frequency (刷新频率) 一般60Hz.



光栅扫描图形显示器的特点

优点:

- ❖ 成本低, 易于绘制填充图形
- ❖ 刷新频率一定, 与图形的复杂程度无关, 易于修改图形

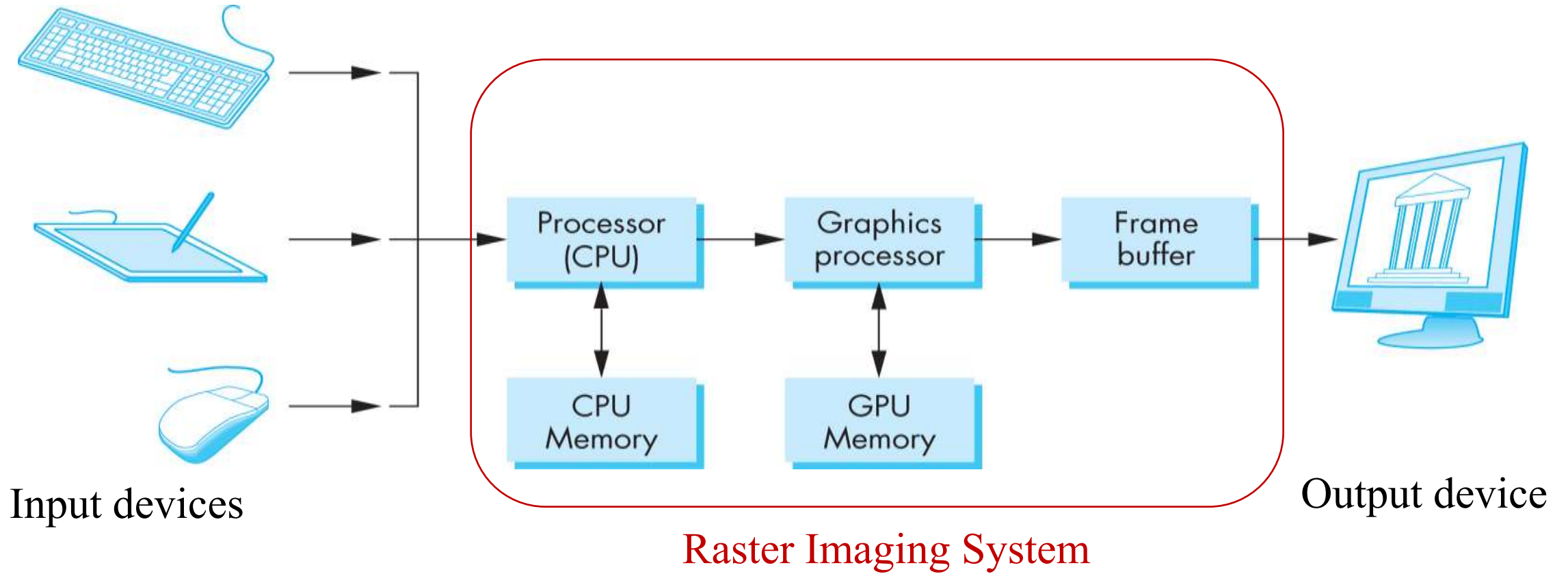
缺点:

- ❖ 需要扫描转换
- ❖ 会产生走样

Computer Graphics System Outlines

- Input devices
- Output device
- Process device* (Raster Imaging System)
- Image Formation Principle –光栅成像原理
- Rendering Pipeline –渲染管线实现*
- The Programmer's Interface-程序员编程接口

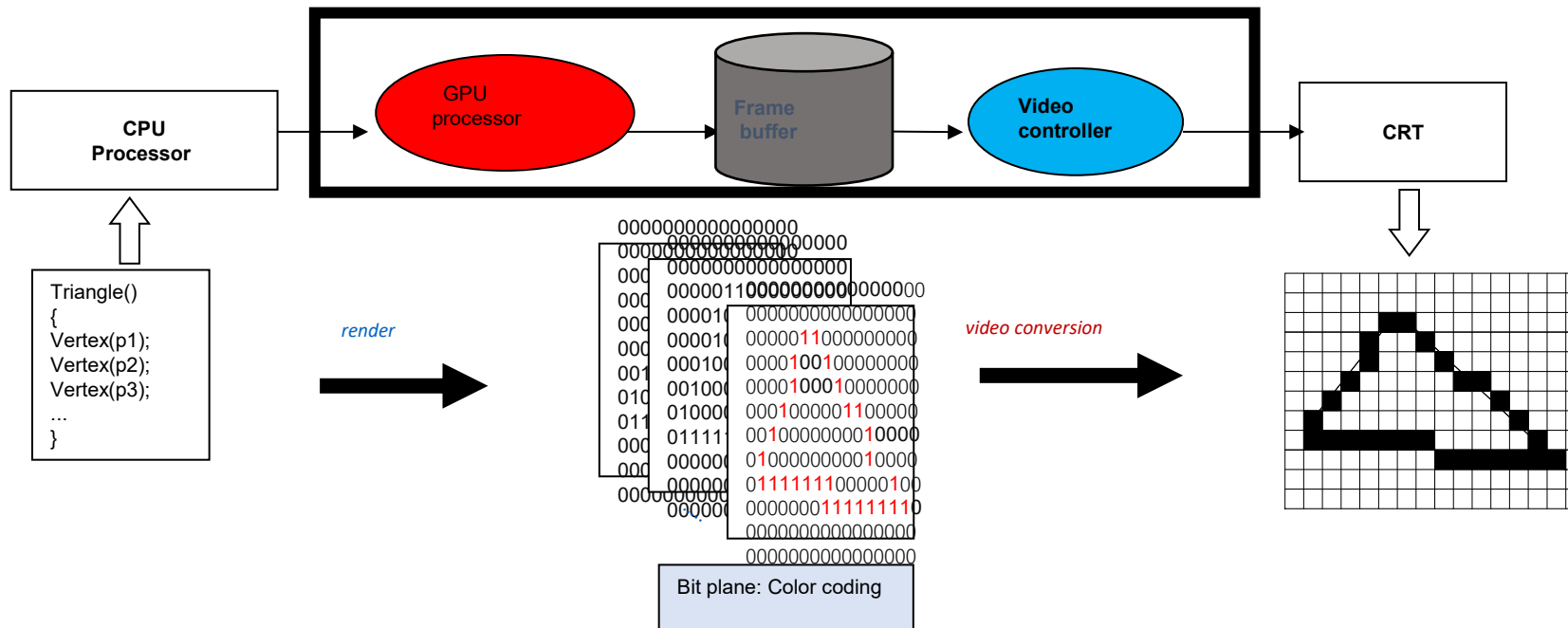
Basic Graphics System



Raster Imaging System

➤ Main constituents (成分)

1. **Controller**视频控制器
2. **Framebuffer**帧缓存
3. **Processor**显示处理器

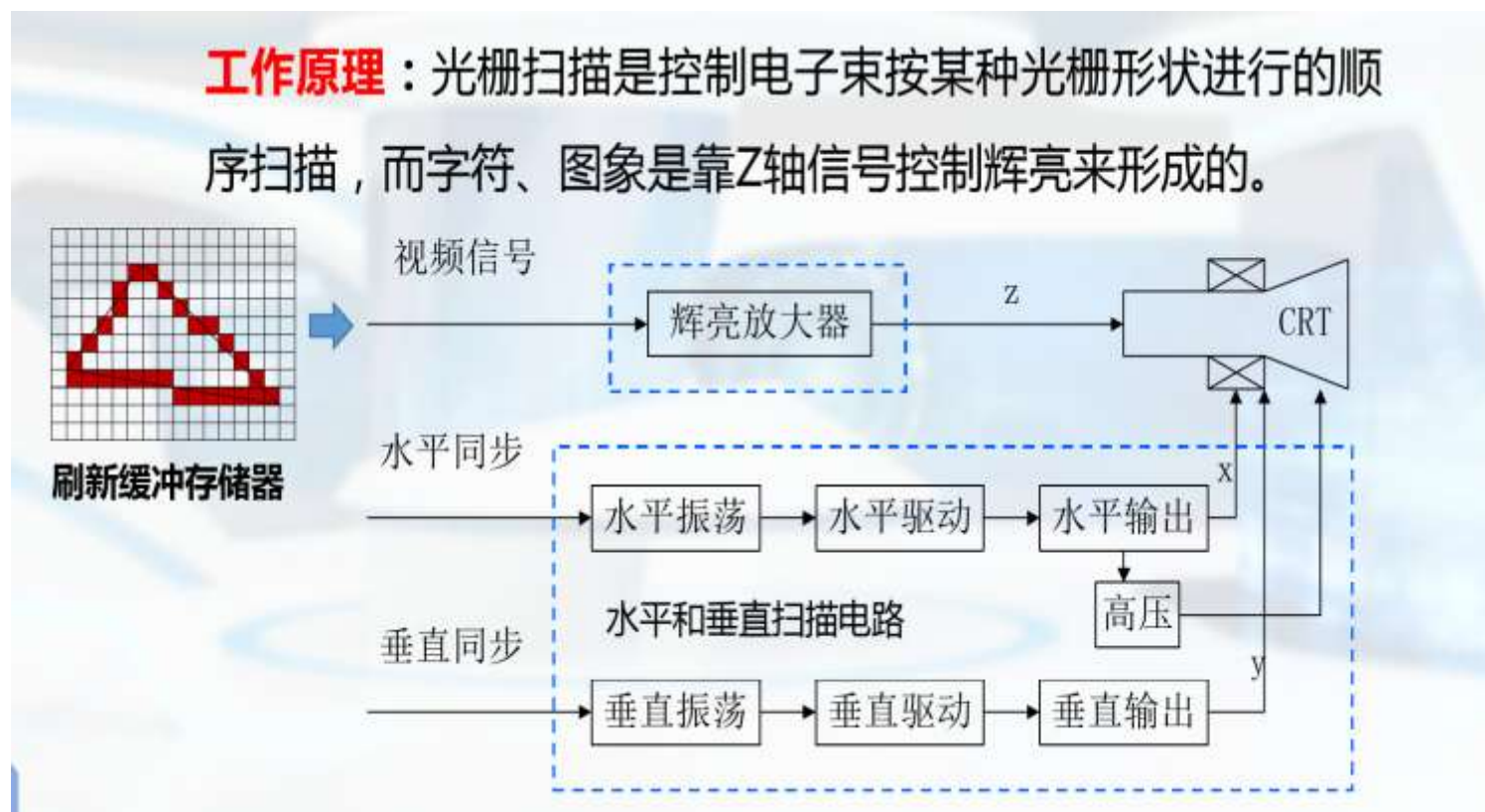


Raster Imaging System(cont.)

1. Video controller 视频控制器

DAC: Digital-to-Analog Converter:

数模转换器：数字信号转换为模拟信号



Raster Imaging System(cont.)

2. Frame Buffer帧缓存:

- a portion of random-access memory (RAM) containing a bitmap that drives a video display.
- It is a memory buffer containing data representing all the pixels in a complete video frame.

//bitmap, pixelmap, rastermap都是指的一帧画面，颜色信息的存储空间就是帧缓存

Question?

- 1. How colors are represented/coded ? (颜色如何表示/编码?)
 - Color Model/Color Space (颜色模型/颜色空间)
- 2. How much capacity does a frame image need (帧缓存需要多大容量?)
 - Framebuffer capacity(帧缓存容量) = Resolution * Pixel Depth
- 3. How color codes are stored in framebuffer? (颜色编码如何存储?)
 - Packed Pixel Method(组合像素法)
 - Bit Plane Method(位面法)
 - Color Lookup Table(CLT查色表)

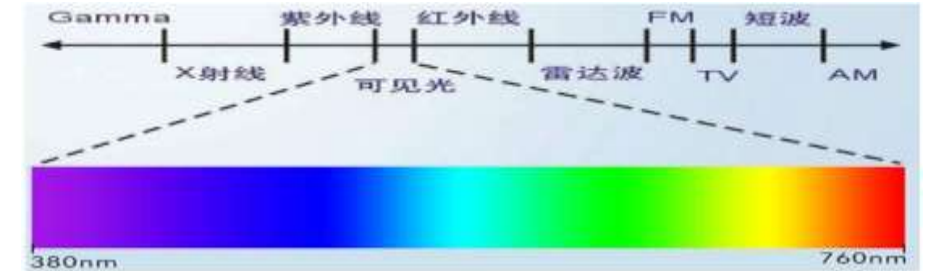
Raster Imaging System(cont.)

2. Frame Buffer帧缓存(cont.)

➤ How colors are represented/coded ? (颜色如何编码表示)

➤ Color & Light

- Light is the part of the electromagnetic spectrum that causes a reaction in our visual systems, Generally these are wavelengths in the range of about 350-750 nm (nanometers)
- Long wavelengths appear as reds and short wavelengths as blues

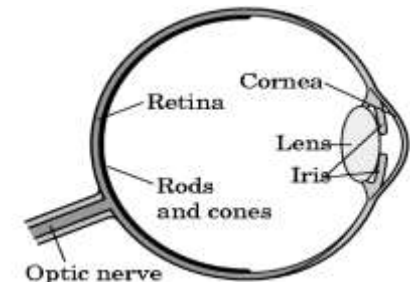


➤ Three-Color Theory (三基色理论)

- Human visual system has two types of sensors
- Rods视杆细胞: monochromatic, night vision
- Cones视锥细胞: Color sensitive, Three types of cones, Only three values (the tristimulus values) are sent to the brain

Need only match these three values

Need only three primary colors



Raster Imaging System(cont.)

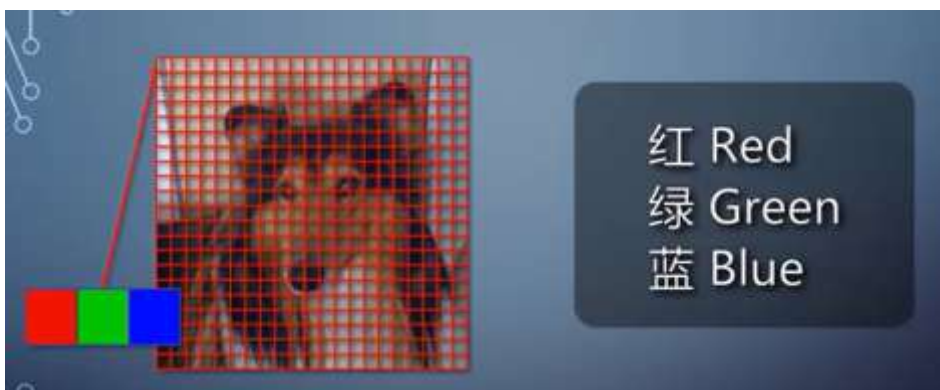
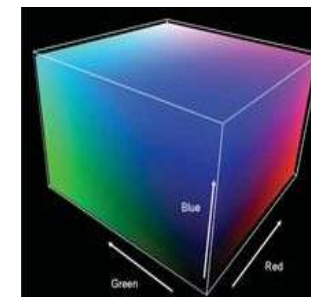
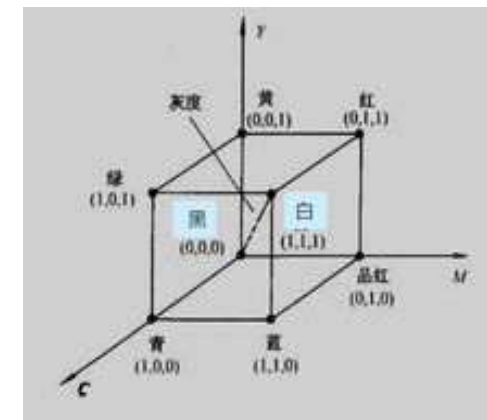
2.Frame Buffer帧缓存(cont.)

How colors are represente/coded ? (颜色如何编码表示)

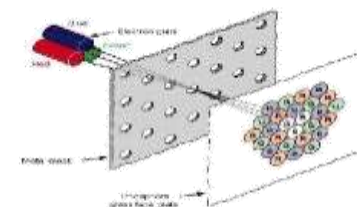
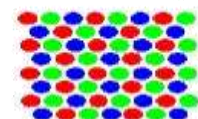
➤ RGB Model/ RGB space: $C=r[R]+g[G]+b[B]$

➤ Color CRT using RGB Color Model

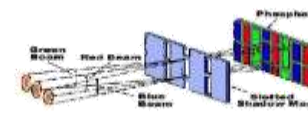
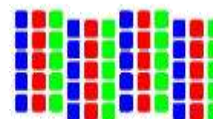
- Red: (1.0, 0.0, 0.0) , Green: (0.0, 1.0, 0.0) , blue(0.0,0.0,1.0)
- White (1.0, 1.0, 1.0) //当三基色分量都为1.0时,混合为白色光
- Black (0.0, 0.0, 0.0) //当三基色分量都为0.0时,混合为黑色 (没有光)



Delta Electron Gun Arrangement



Tri-line Electron Gun Arrangement



Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

How colors are represente/coded ? (颜色如何编码表示?)



➤ CMY model: Subtractive color

➤ 打印机使用的是CMYK（减色法）模型用墨水吸收光线来呈现颜色。

➤ C (Cyan - 青色)：吸收红色，反射蓝色和绿色。

➤ M (Magenta - 品红色)：吸收绿色，反射红色和蓝色。

➤ Y (Yellow - 黄色)：吸收蓝色，反射红色和绿色。

➤ K: 在CMY的基础上增加了 K (Key plate - 关键色板) 也就是黑色。

理论上，将CMY三种颜色100%混合应该能产生黑色。但现实中这几乎不可能。

Form a color by filtering white light with cyan (C), Magenta (M), and Yellow (Y) filters

Light-material interactions, Printing（印刷）, Negative film(底片)

- **white (0.0, 0.0, 0.0), black (1.0, 1.0, 1.0)**
- **Red (0.0, 1.0, 1.0) , Green(1.0, 0.0, 1.0), blue(1.0,1.0,0.0)**
- **Cyan (1.0, 0.0, 0.0) , Magenta(0.0, 1.0, 0.0), Yellow(0.0,0.0,1.0)**

Raster Imaging System(cont.)

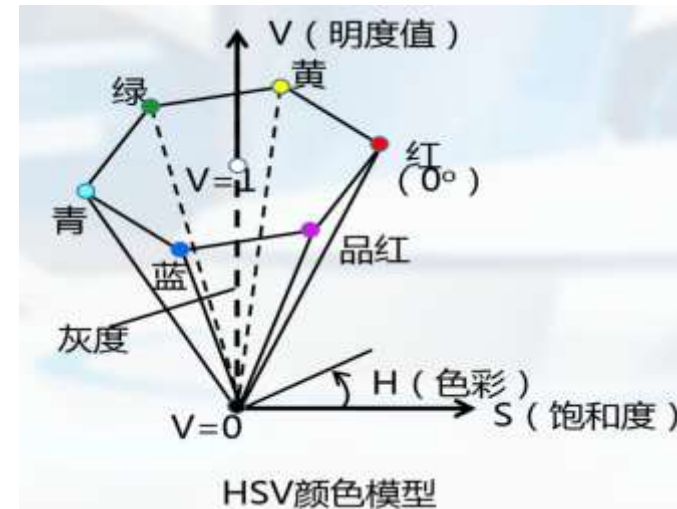
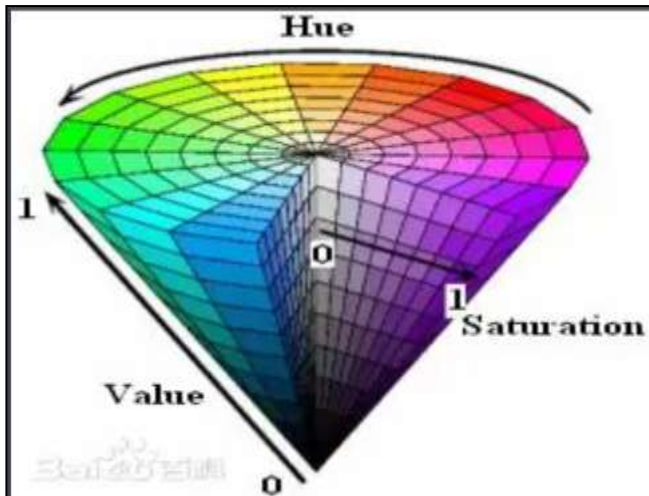
2.Frame Buffer帧缓存(cont.)

How colors are represente/coded ? (颜色如何编码表示)

- HSV/ HSL/HSB/HSI model

- 适合人类进行表达的颜色模型

- HSV: 色调Hue , 饱和度Saturation , 亮度Value
 - HSL: Hue, Saturation, Lightness/ Luminance /Bright/ Intensity

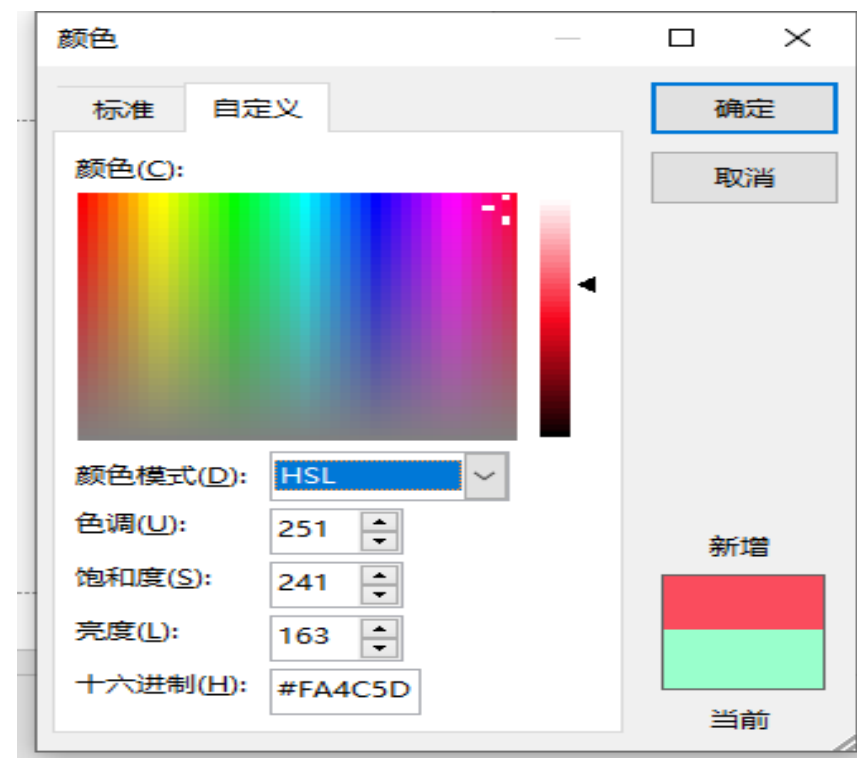
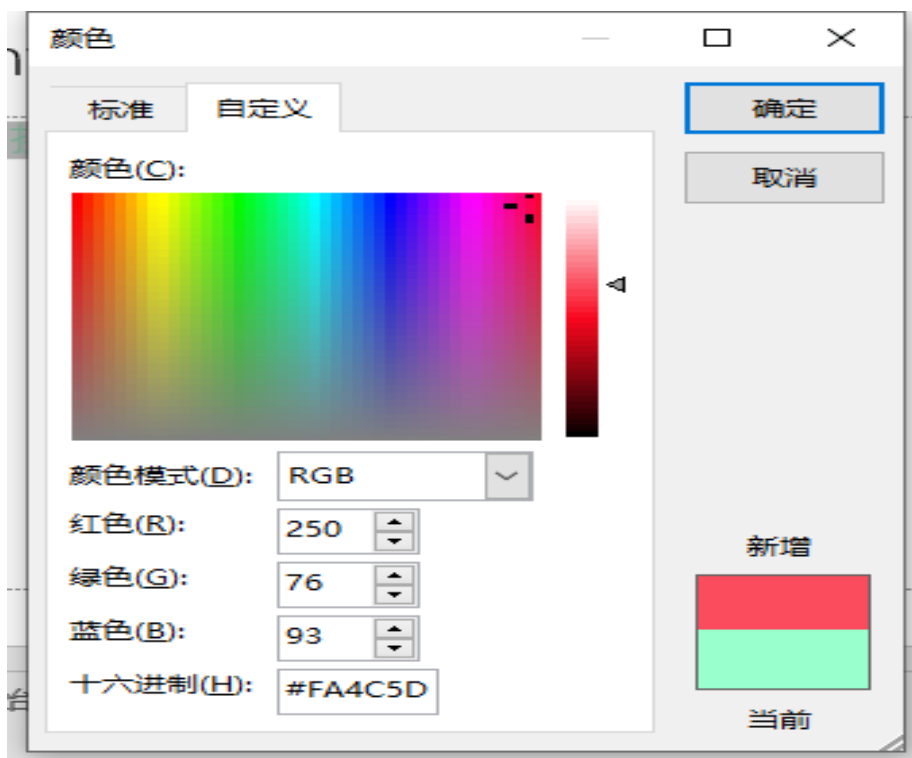


Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

How colors are represente/coded ? (颜色如何编码表示)

✓颜色模型表示之间可以相互转换： //注意这里的编码是整数： 0到255



Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

➤ How much capacity does a frame image need (帧缓存需要多大容量?)

➤ **Framebuffer capacity = Pixel Precision * Resolution**

- **Pixel Depth 像素深度 or Pixel Precision 像素精度**

- Defined as the number of bits that are used for each pixel,
- which determines properties such as how many colors can be represented on a given system

Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

- How much capacity does a frame image need (帧缓存需要多大容量?)

□ Monochrome(黑白图像): 1 bits Per Pixel

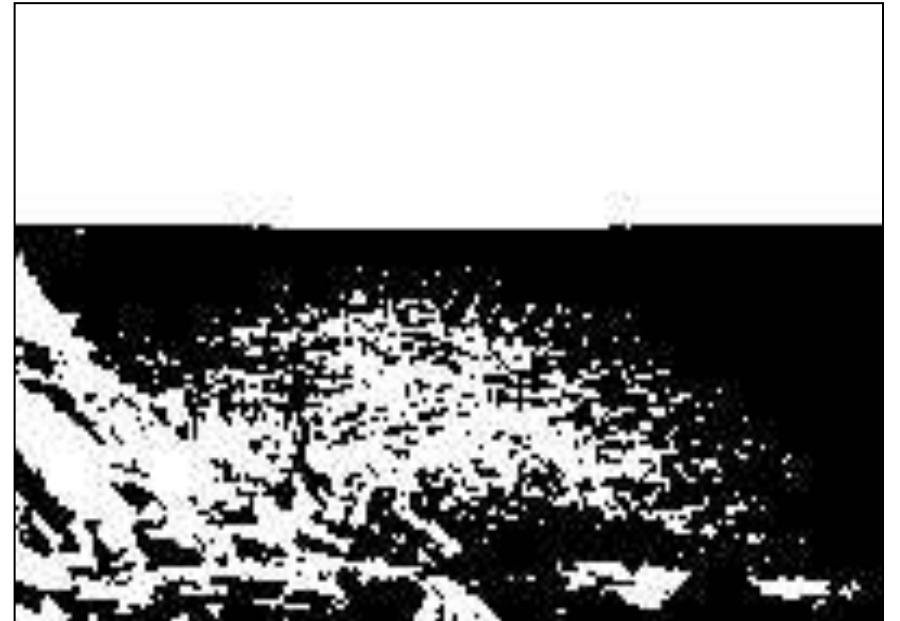
- Each color component is represented by 1 bit.
- pure **black** or pure **white**

Pixel Precision: 1

Supply Colors: $2^1=2$

If Resolution is : $1024*768$

Framebuffer Capacity: $1*1024*768$ bit



Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

➤ How much capacity does a frame image need (帧缓存需要多大容量?)

□ Gray-scale Image灰度图像: 8 bits per pixel

No colours besides black, white and grey

Analogous to working with black and white film or television

Pixel Precision: 8

Supply Colors: : $2^8=256$

If resolution is: $1024*768$

Framebuffer Capacity: $8*1024*768$ bit



Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

➤ How much capacity does a frame image need (帧缓存需要多大容量?)

- **Bit Colour位色图像: 8 bits per pixel**

Each pixel may composes of red-3bit, green-3bit and blue-2bit.

Pixel Precision: 8

Supply Colors: $2^8=256$

If resolution is : $1024*768$

Framebuffer Capacity: $8* 1024*768$ bit



Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

➤ How much capacity does a frame image need (帧缓存需要多大容量?)

□ True color真彩色图像: 24 bits per pixel

➤ Each pixel composes of red, green and blue (3 components).

➤ Each color component is represented by 1 byte (8 bits).

Pixel Precision: $8 \times 3 = 24$

Allows Colors : $2^{24} = 16\text{M}$

If resolution is : 1024×768

Framebuffer Capacity: $24 \times 1024 \times 768$ bit

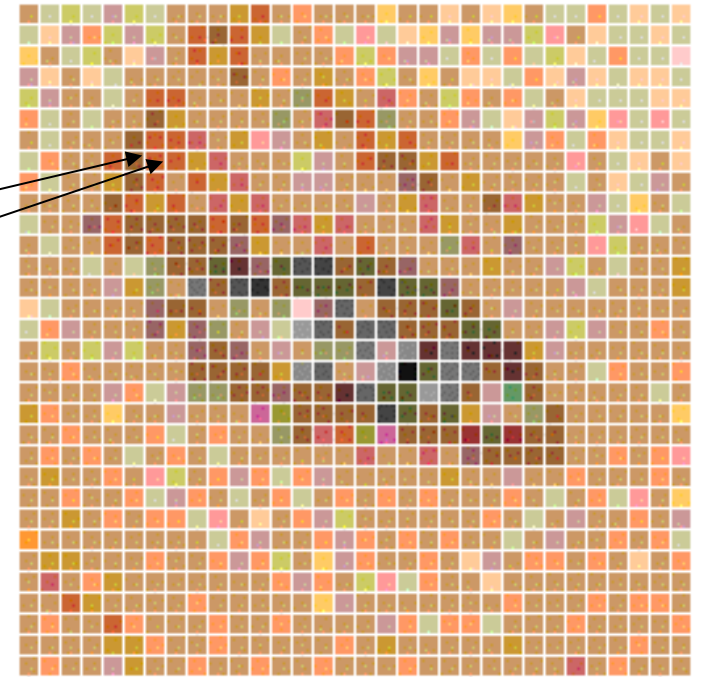
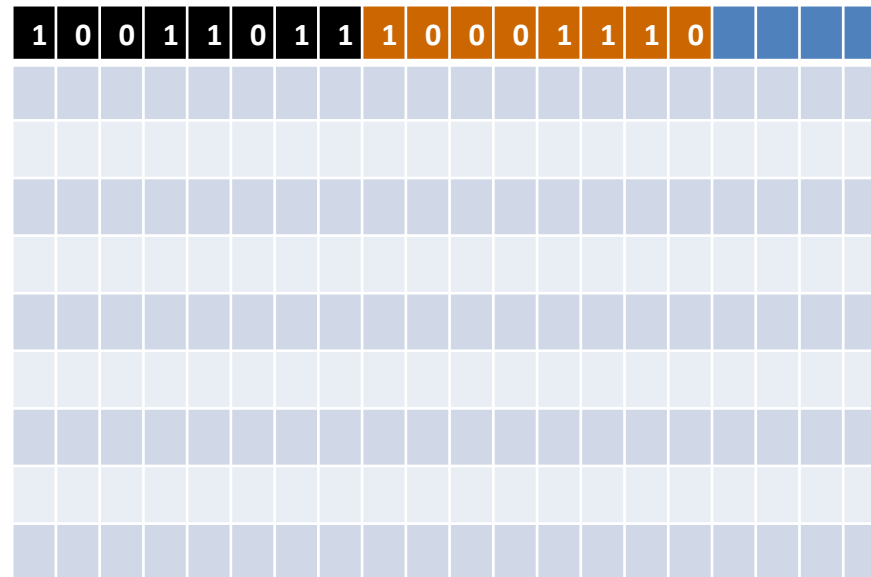


Raster Imaging System(cont.)

➤ 2.Frame Buffer帧缓存(cont.)

➤ How color codes are stored in framebuffer? (颜色编码如何存储?)

□ Color Storage: Packed Pixel Method(组合像素法)



screen

Frame buffer

Raster Imaging System(cont.)

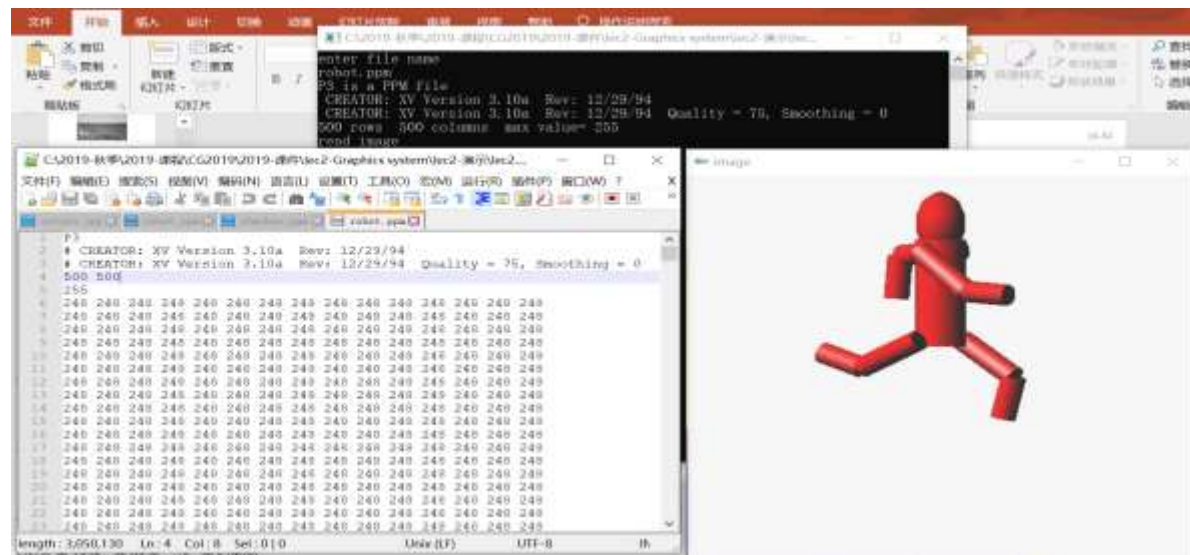
2.Frame Buffer帧缓存(cont.)

How color codes are stored in framebuffer? (颜色编码如何存储)

□Color Storage: Packed Pixel Method(组合像素法): 一般是图像文件中的组织方式

□Example: PPM file (Portable Pixmap)

- Resolution:500*500
- RGB Color Model, 3 components and each occupy one byte, color value:0~255



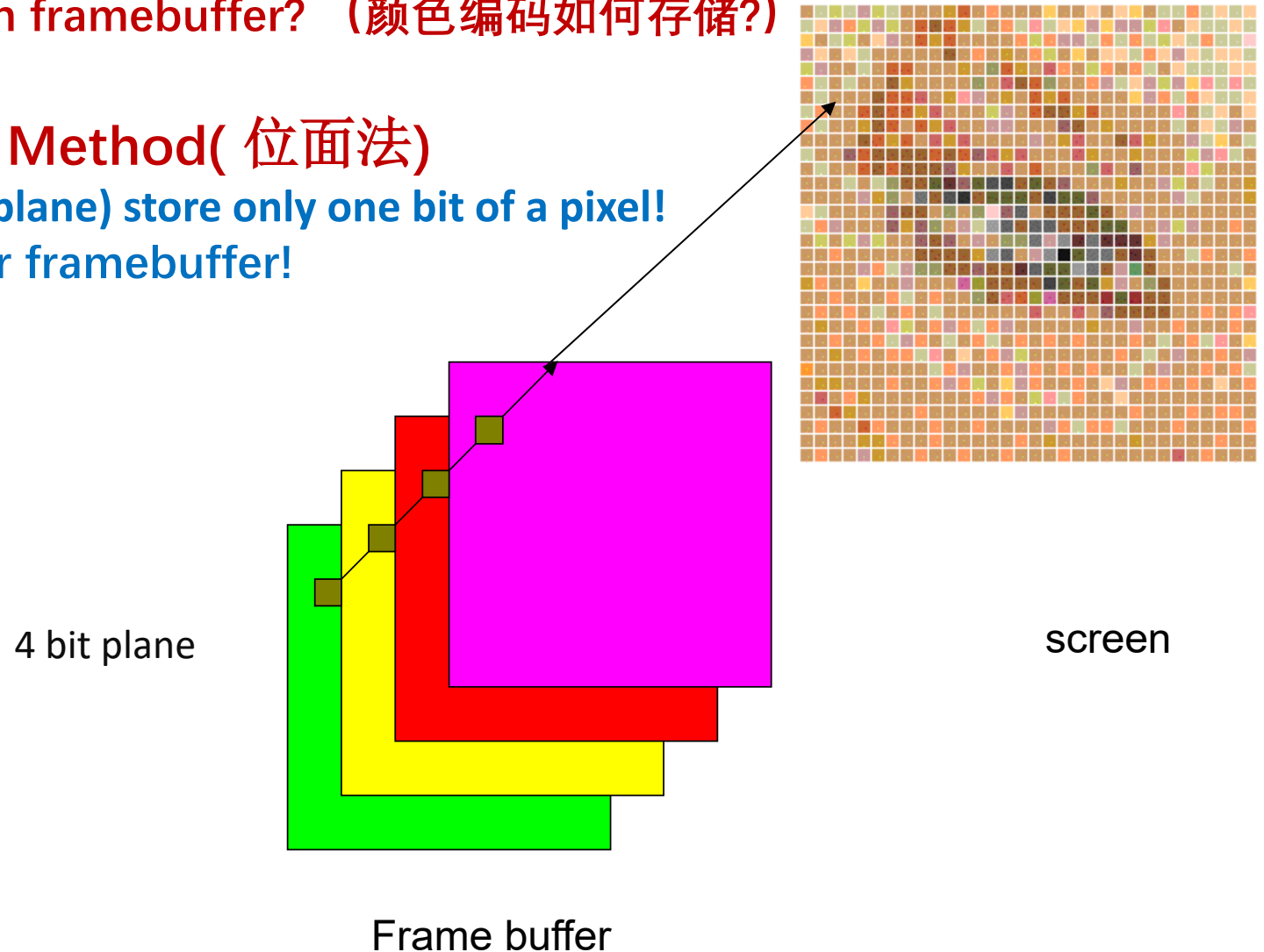
Raster Imaging System(cont.)

➤ 2.Frame Buffer帧缓存(cont.)

➤ How color codes are stored in framebuffer? (颜色编码如何存储?)

□ Color Storage : Bit Plane Method(位面法)

- bit plane(位面):One unit(bit plane) store only one bit of a pixel!
- This Method is chosen for framebuffer!



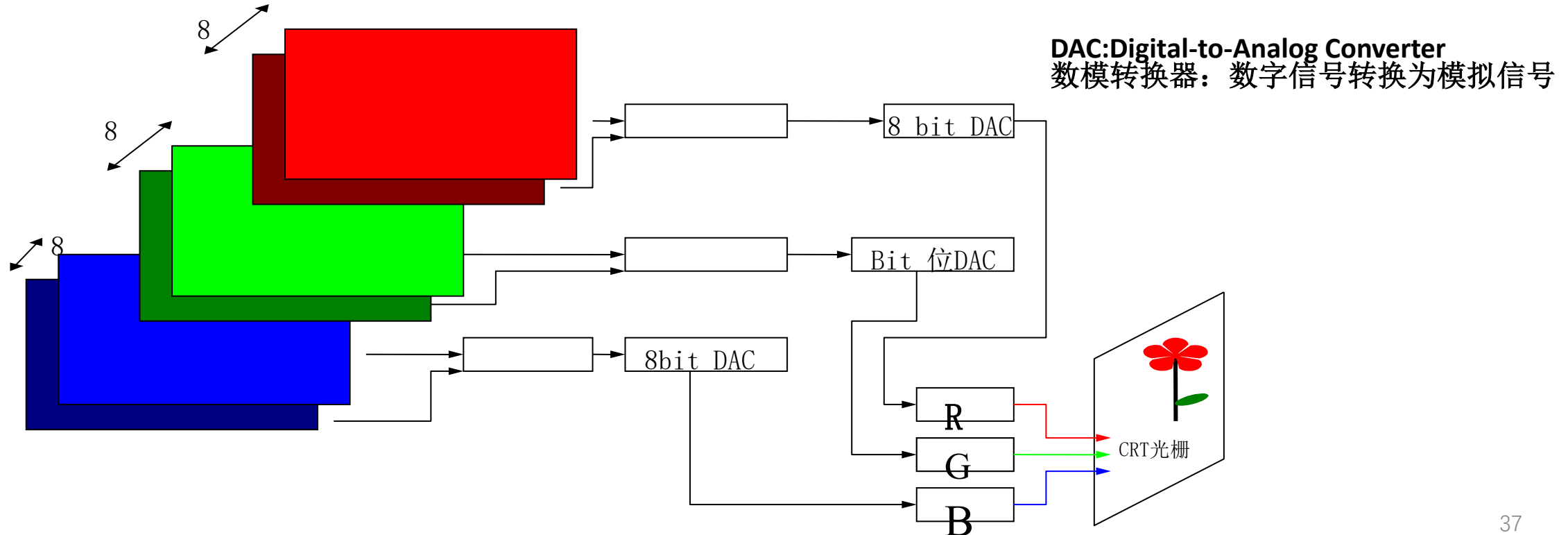
Raster Imaging System(cont.)

2.Frame Buffer帧缓存(cont.)

How color codes are stored in framebuffer? (颜色编码如何存储)

□ Color Storage : Bit Plane Method(位面法): 帧缓存中的存储方式

□ Example : True color system has 24 bit planes to store color codes



Raster Imaging System(cont.)

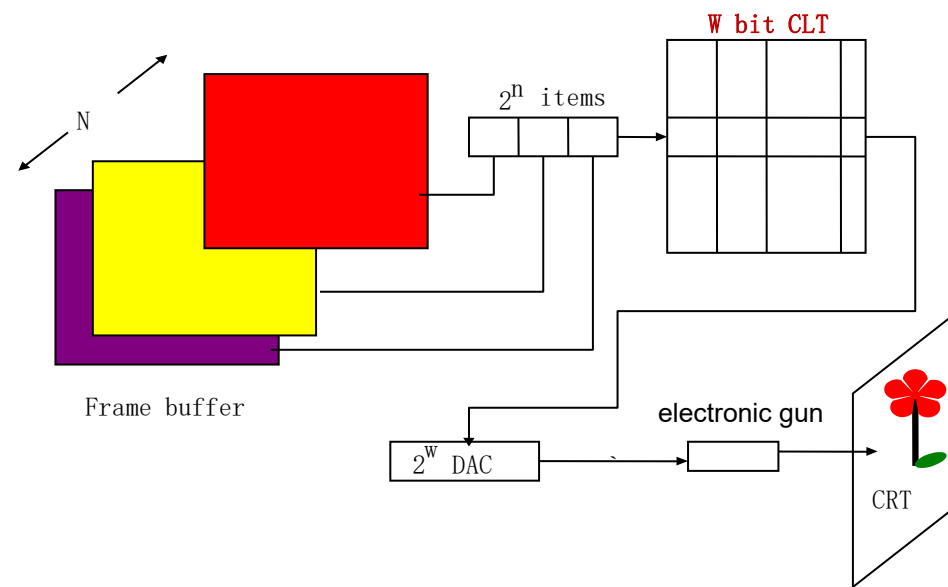
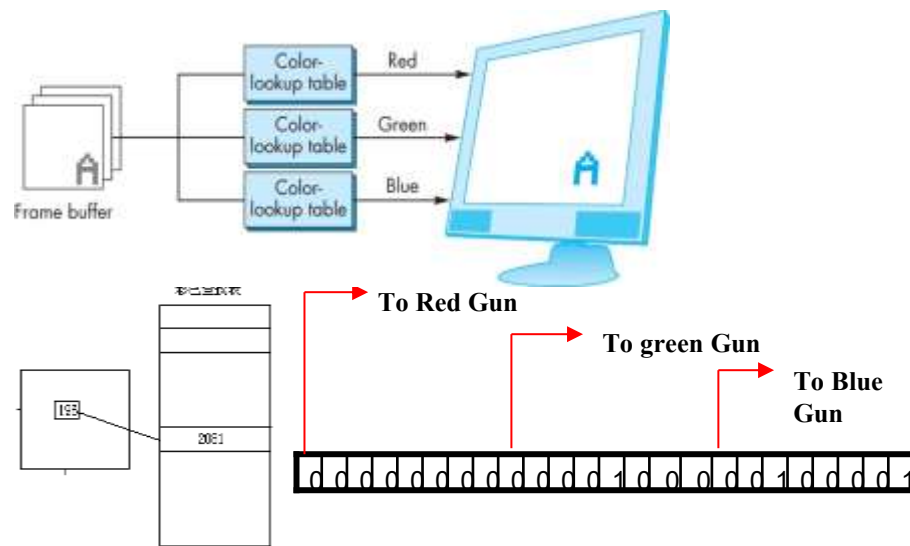
2.Frame Buffer帧缓存(cont.)

How color codes are stored in framebuffer? (颜色编码如何存储?)

□ Color Lookup Table(CLT查色表)

- 帧缓存存查色表的索引地址, 真正的颜色编码RGB存储在查色表CLT中

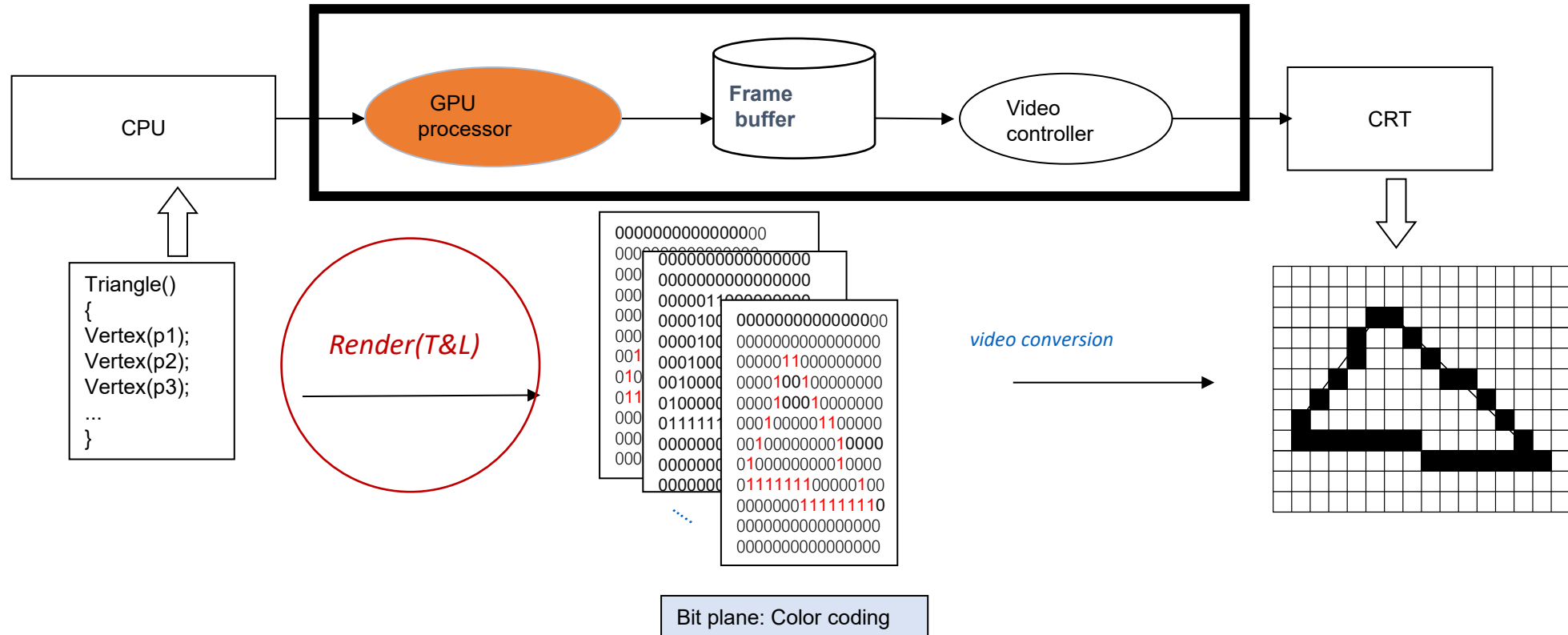
- Framebuffer store Indices(索引): Indices usually 8 bits,
- CLT store true color: has 256 items and each item has 24bit color code.



Raster Imaging System(cont.)

➤ 3.显示处理器GPU

A **graphics processing unit (GPU)** is a specialized electronic circuit designed to manipulate and alter memory to accelerate the creation of images in a frame buffer intended for output to a display device.



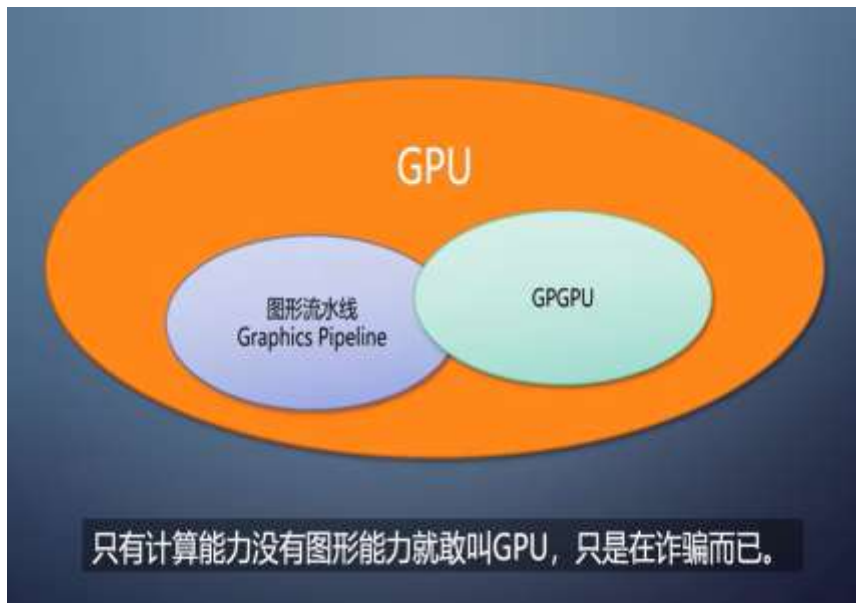
Raster Imaging System(cont.)

3.显示处理器GPU(cont.)

为什么GPU（graphics processing unit）适合作图形处理？

➤CPU vs. GPU架构差异：

- CPU是“低延迟”设计（擅长快速完成**单一**的**复杂任务**），
- GPU是“高吞吐”设计（拥有成千上万个核心，擅长**并行**处理大量**简单任务**）



GPU 主芯片

特点：擅长计算，在矩阵运算、运算密集型任务方面更有优势

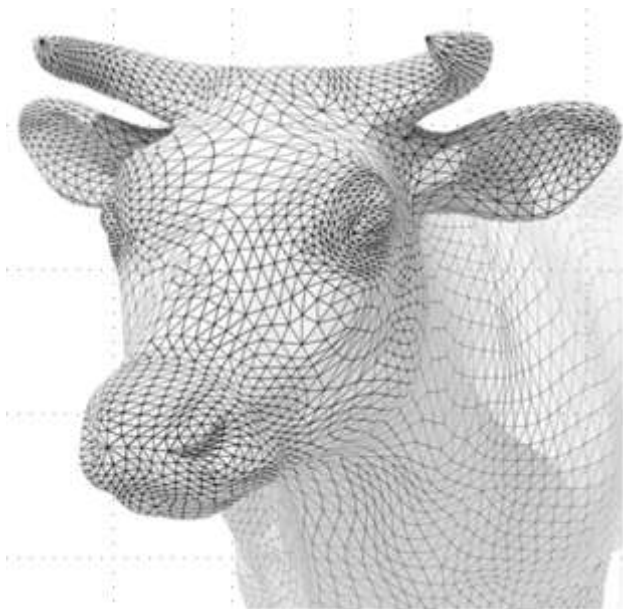
应用：计算机图形学、计算机视觉、机器学习、深度学习、图像处理、高性能计算等多个领域。



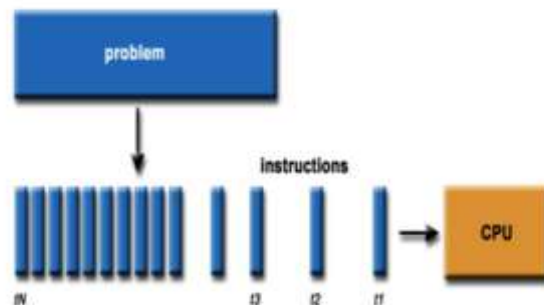
Raster Imaging System(cont.)

3.显示处理器GPU(cont.)

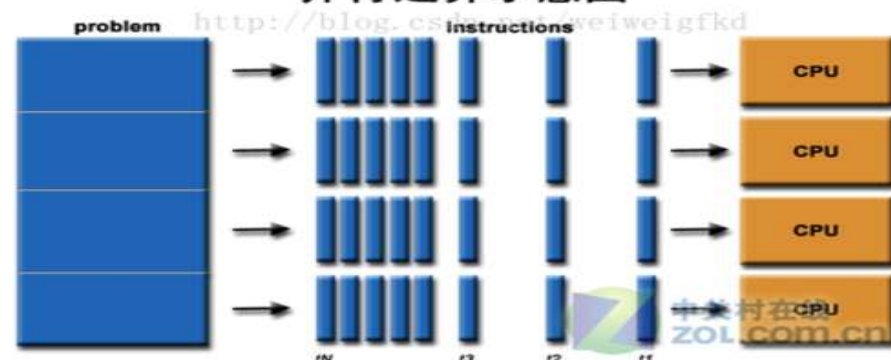
- CPU的单个“强核”，“串行”处理每个数据（顶点或像素）
- GPU的多个“弱核”，“并行”处理每个数据（顶点或像素）
- 举例类比：餐馆的外卖采用“多个小摩托车”运 比 “单个大货车运” 运 要快， 客户等候时间短
 - CPU型餐馆用一辆大货车依次送货， 每次可以拉很多外卖， 但是送完一家才能到下一家送货， 每个人收到外卖的时间必然很长；
 - 而GPU型餐馆用多辆小摩托车同时送货， 每辆车送出去的不多， 但是并行处理的效率高， 点餐之后收货就会比大货车快很多



串行运算示意图



并行运算示意图



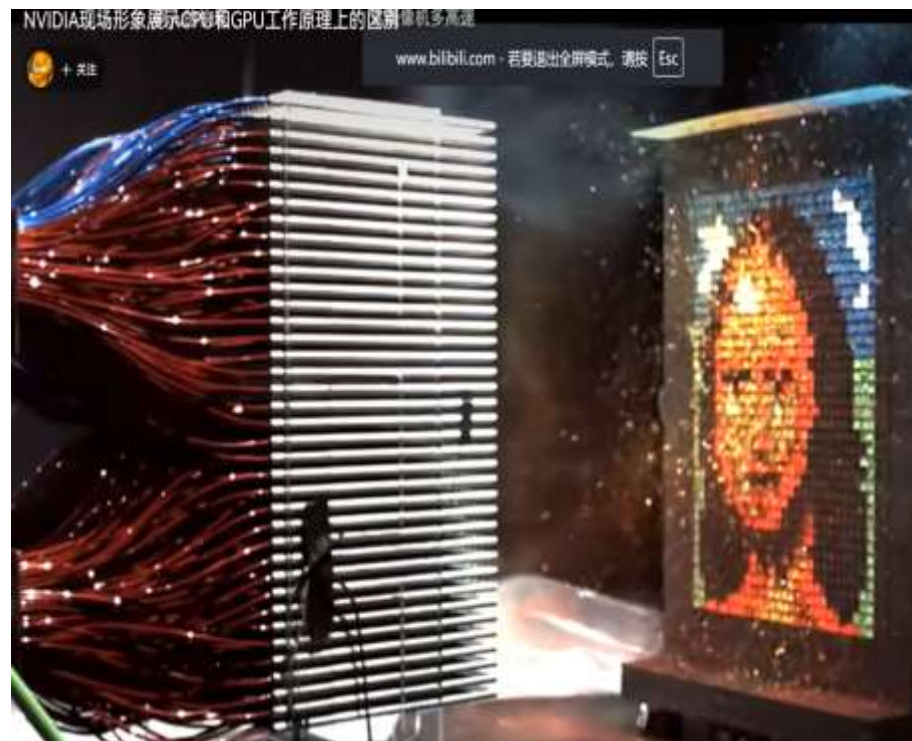
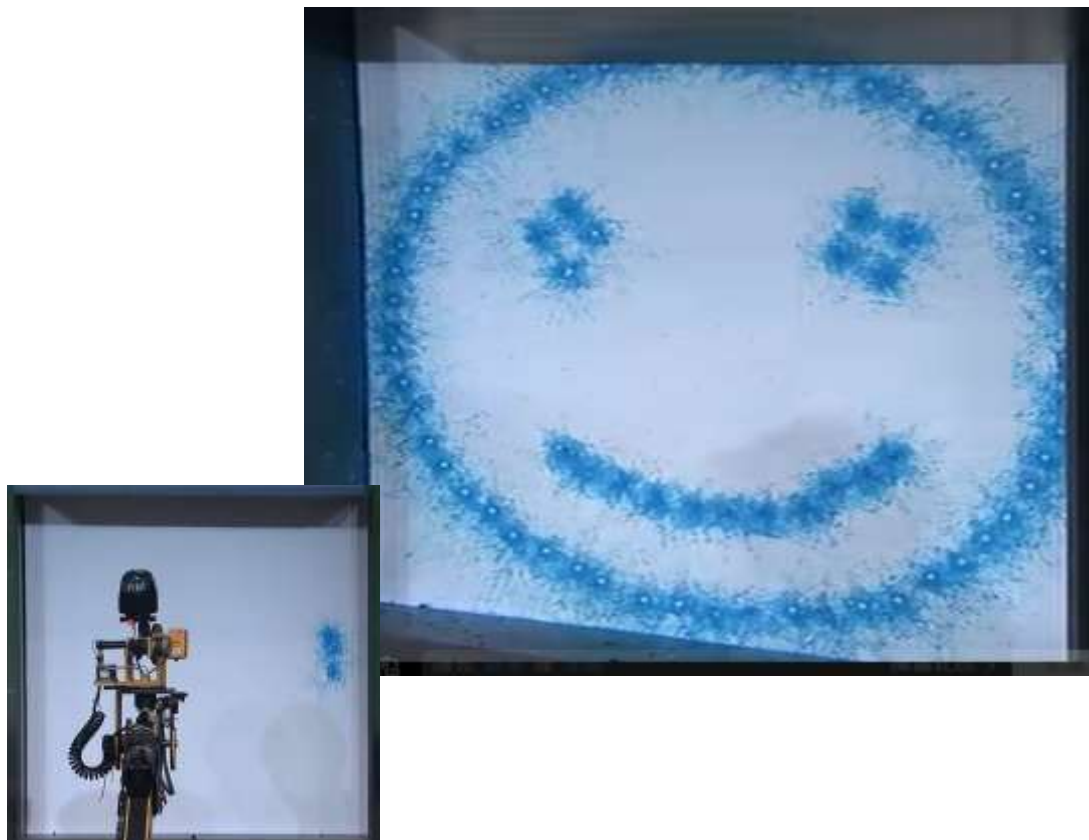
Raster Imaging System(cont.)

- NVIDIA现场形象展示CPU和GPU工作原理上的区别

https://www.bilibili.com/video/BV1ry4y1y7KZ/?spm_id_from=333.337.search-card.all.click

左边：CPU串行绘制简单图像而用时长

右边：GPU并行绘制复杂图像而用时1秒



Raster Imaging System(cont.)

- 例：当NVIDIA宣传RTX 4090有“18432个CUDA核心”时，指的是芯片内部集成了超过1.8万个专门用于并行计算的单元。

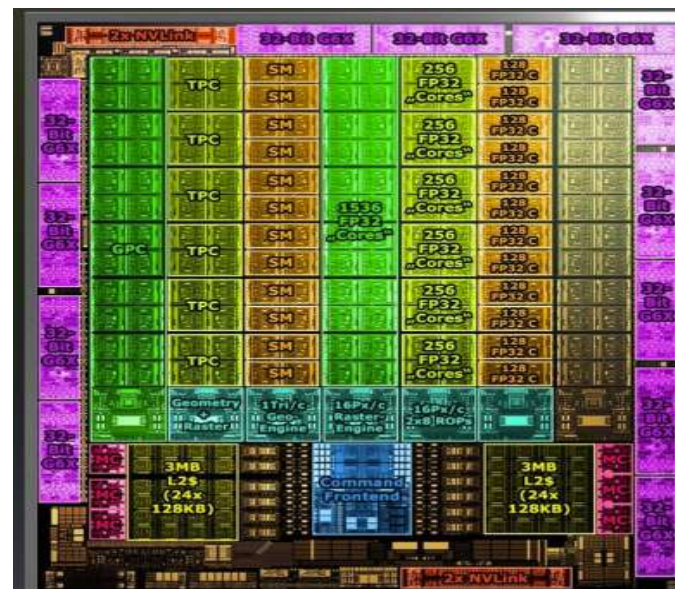
➤NVIDIA的AD102芯片（用于RTX 4090）是一个GPU芯片。

- 这个芯片内部包含了多达144个流式多处理器，
- 每个流式多处理器内部又包含 128个CUDA核心。

那么，该GPU芯片总的CUDA核心数可以达到 $144 \text{ SM} \times 128 \text{ 核心/SM} = 18,432 \text{ 个CUDA核心}$ 。



英伟达ad102:引领gpu行业的性...



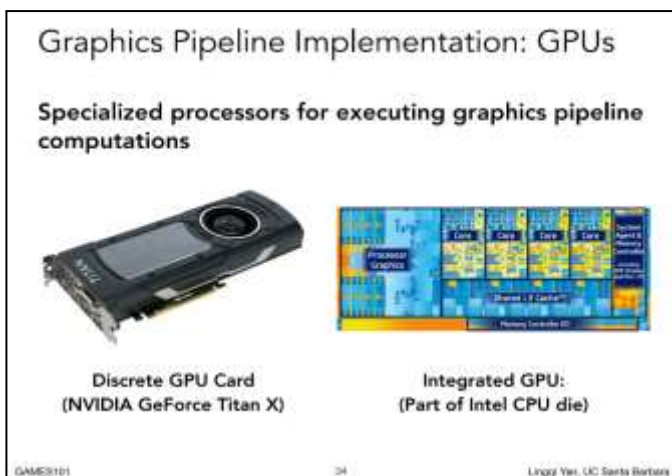
Raster Imaging System(cont.)

3.显示处理器GPU(cont.)

➤ 显卡：集成显卡，独立显卡

➤ 独立显卡：NVIDIA（英伟达）显卡发展史

- 1999, GeForce 256, NVIDIA(英伟达公司) 首次推出GPU
- 2003, GeForce 5800 Ultra, shader programming!
- 2018, RTX 2080, Turing GPU, flagship: support real-time ray tracing in games! Using DLSS to scale to 4K, A hybrid rasterization and ray traced demo,
- 2023, Racer RTX, starting “generative AI era”, (ref: SIGGRAPH2023)
- 2024/2025年, GeForce RTX 50系列 (如 RTX 5090, RTX 5080)



Raster Imaging System(cont.)

3.显示处理器GPU(cont.)

主要应用领域：

- 图形渲染（本职工作）：处理3D游戏、影视特效、CAD设计等中的顶点和像素计算。
 - 视频编解码：对视频流的每一帧进行并行处理。
 - 人工智能与深度学习（当前最火的应用）：神经网络训练和推理的核心是矩阵乘法和卷积，高度并行，完美契合GPU架构。
 - 科学计算与仿真：如流体动力学、天气预测、分子建模等。
 - 密码学与数据加密：暴力破解和加密算法涉及大量并行计算。
- 总而言之，GPU的特点使其从专用于图形的处理器，演变成了如今驱动人工智能和科学发现的通用并行计算引擎。

Computer Graphics System Outlines

- Input devices
- Output device
- Process device*
- **Image Formation Principle –成像原理***
- Rendering Pipeline –渲染管线*
- The Programmer's Interface-应用编程接口API*

图形的软件层次

层级	软件模块/代表	核心职责（它在干什么）	对硬件架构的依赖/适配
应用层	游戏引擎 (Unity/Unreal) CAD/影视软件	场景管理与剔除：准备模型、贴图、光源数据； 计算哪些物体在摄像机视野内（视锥剔除）。	几乎无关。此层只关注逻辑，不关心GPU是NVIDIA还是AMD。
API接口层	DirectX 12 / Vulkan Metal / OpenGL	命令翻译与提交：将CPU端的数据打包成GPU能识别的DrawCall和管线状态（PSO），通过驱动发给硬件。	中度相关。现代API（DX12/Vulkan）为了适配多核CPU架构，强制开发者自己管理命令缓冲区（Command Buffer）。
GPU驱动层	NVIDIA/AMD/Intel 驱动 （用户态部分）	着色器编译（JIT/AOT）：将HLSL/GLSL代码编译成当前具体GPU架构（如Ada架构）的微码（ISA）。	高度相关。不同架构的流处理器（SM/CU）布局不同，驱动生成的微码优化策略完全不同。
内核态与固件	内核模式驱动 (KMD) GPU板载固件 (Firmware)	显存管理（虚拟地址映射/交换）与硬件调度：将线程块（Thread Block）分配到具体的计算单元（SM/CU）上，并动态调频。	深度绑定。必须根据架构的缓存大小和显存位宽来调整任务分配粒度（Wavefront/Warp大小）。

Image Formation Principle

- 需要作哪些操作才能生成一帧画面？（如何渲染rendering？）

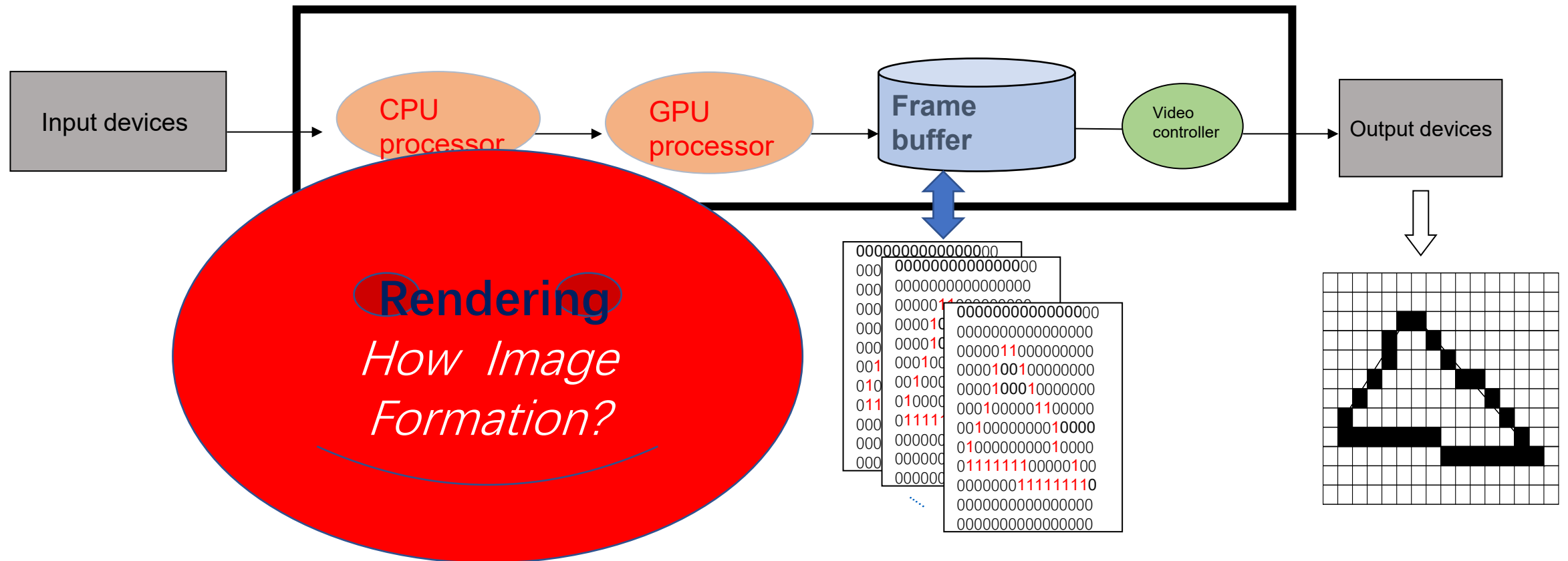


Image Formation Principle

1. Nature Image System: Human visual system

receiving light and transforming it into electrical energy light reflects from objects.

- 物体表面光线进入“人眼”“晶状体”，像是人眼对进入视神经的信号的主观感知！
- 形成有“景深”感（depth of field）的“倒”像，靠感知位“正”！

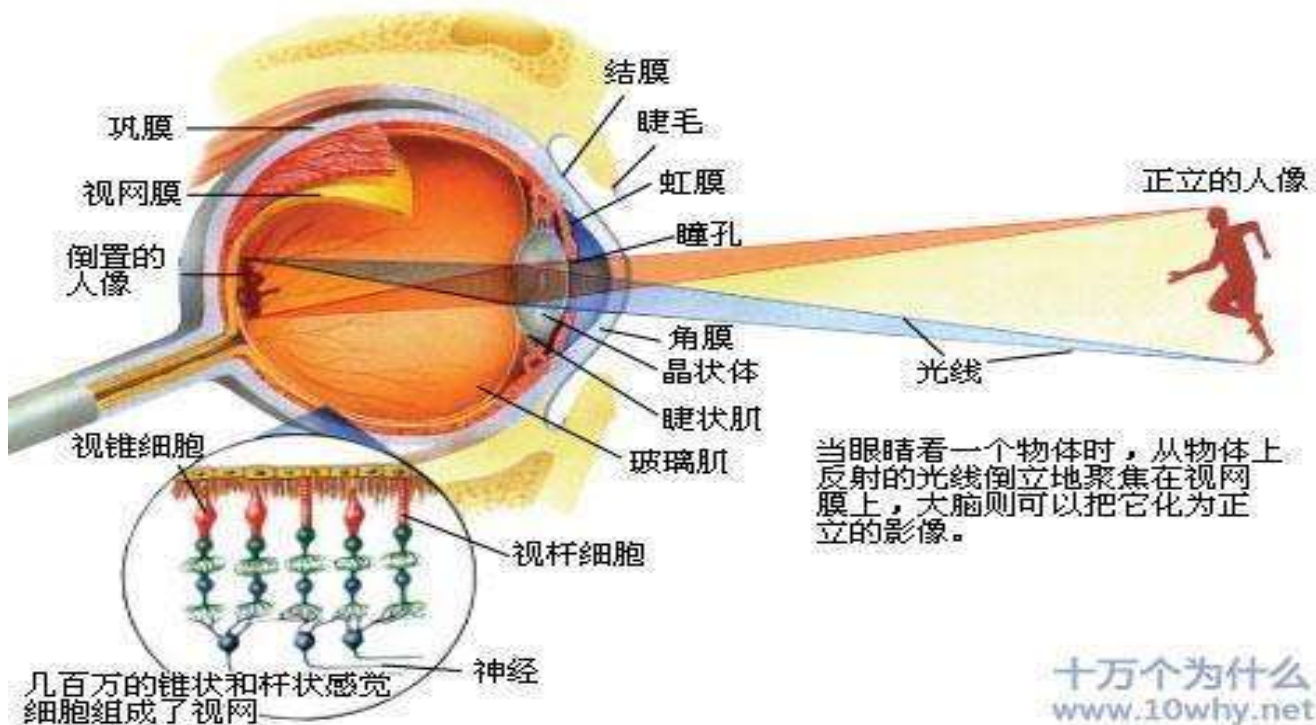
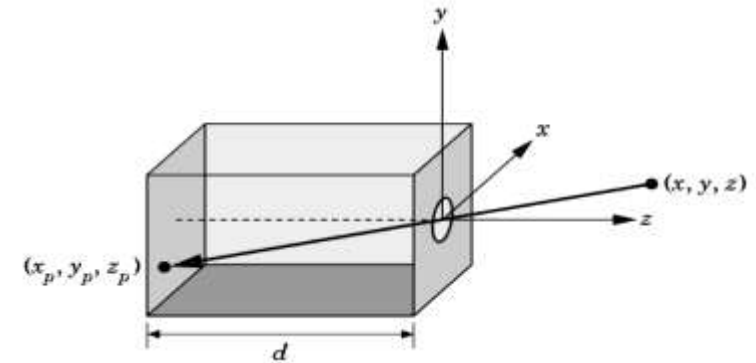
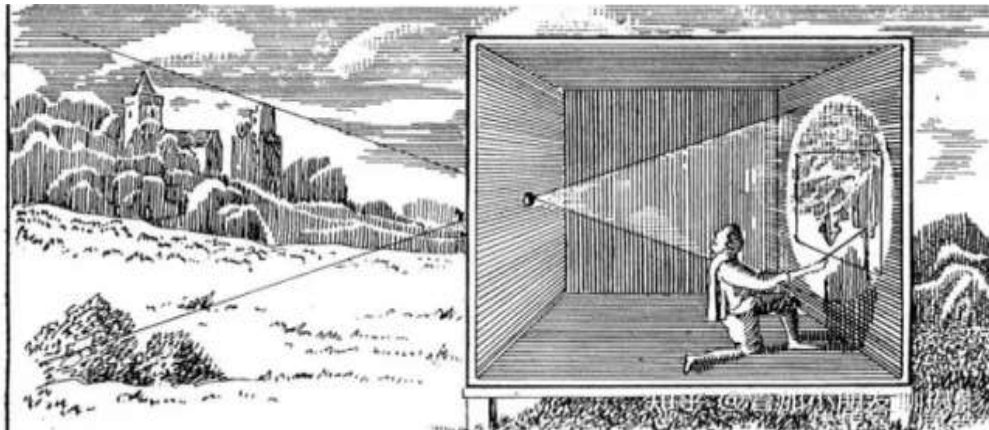


Image Formation Principle(cont.)

2.Pinhole Image Formation(小孔成像)

- Use trigonometry (三角法) to find projection (x_p, y_p, z_p) of point at (x, y, z) , These are equations of simple perspective (简单透视)
- 物体表面光线进入穿过小孔, 在投影平面成“倒”像, “无景深”效果!



$$x_p = -x/z/d \quad y_p = -y/z/d \quad z_p = d$$

Image Formation Principle(cont.)

场景Scene由对象和光源构成，同一场景因观察者不同而成像不同。

Three Elements of Image Formation:

- 对象: Objects (物体对象)
- 光源: Light sources (光源)
- 观察者: Viewer/Camera (观察者/相机)

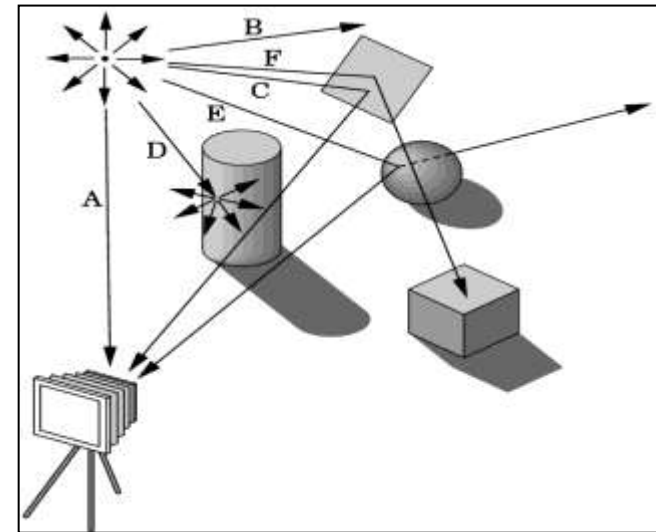
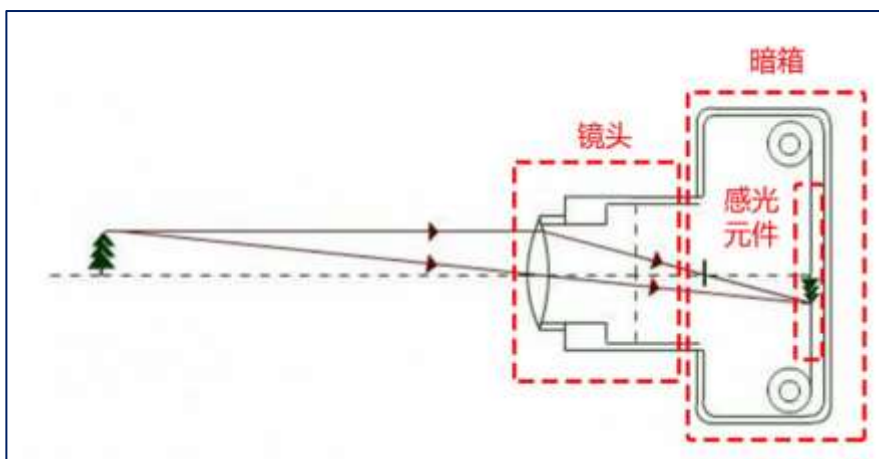
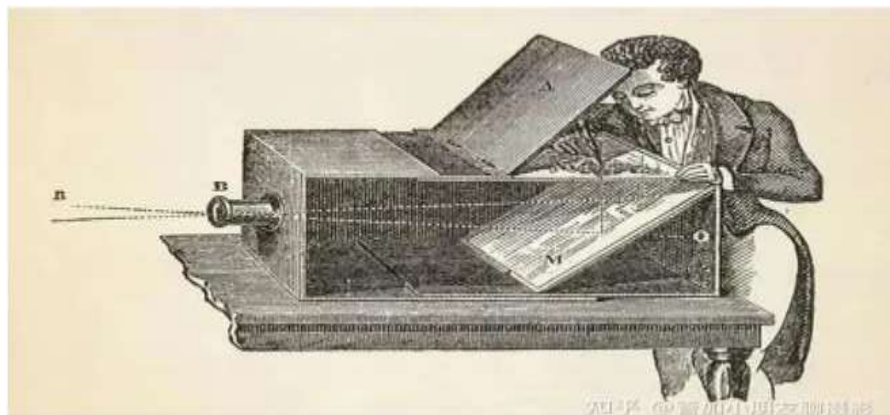
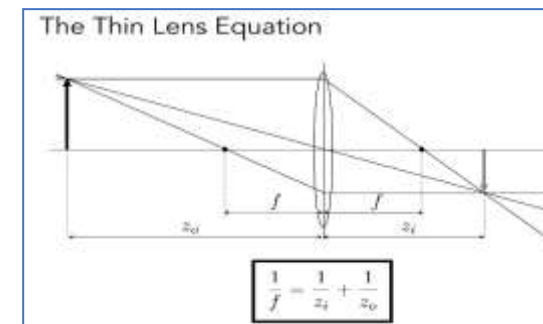


Image Formation Principle(cont.)

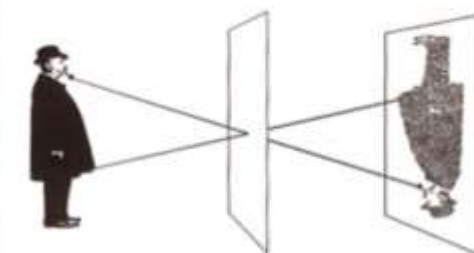
3.Camera imaging system(相机成像)

- 有“棱镜”，产生有“景深”效果的“倒”像
 - *Ref: GAMES101_Lecture_19 Cameras, Lenses and Light Fields*

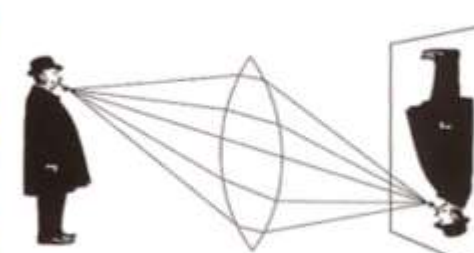


Pinholes & Lenses Form Image on Sensor (传感器)

Photograph made with small pinhole



Photograph made with lens

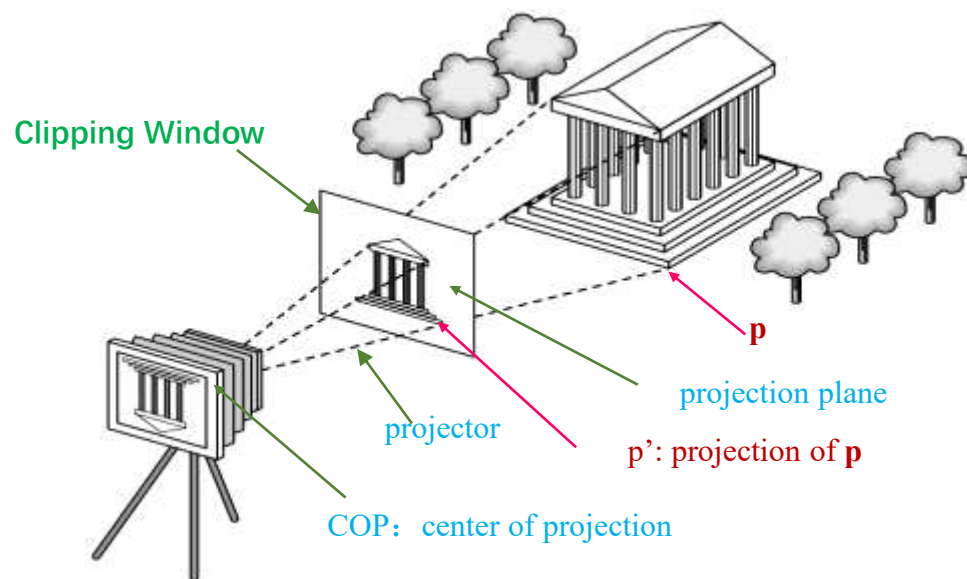


London and Upton

Image Formation Principle(cont.)

4.Synthetic Camera Model 虚拟相机模型/合成相机模型

- 1.对象，光源不依赖于观察者(相机)，三要素相互独立。
- 2.可以如针孔相机一样，利用简单的几何方法计算图像（无景深）
- 3.为了达到正图像，我们在透镜（投影中心点COP)前面设另一个平面作为投影面(Project Plane)
- 4.图像大小是受限的，通过在投影平面内设置一个裁剪矩形(裁剪窗口)加以限制

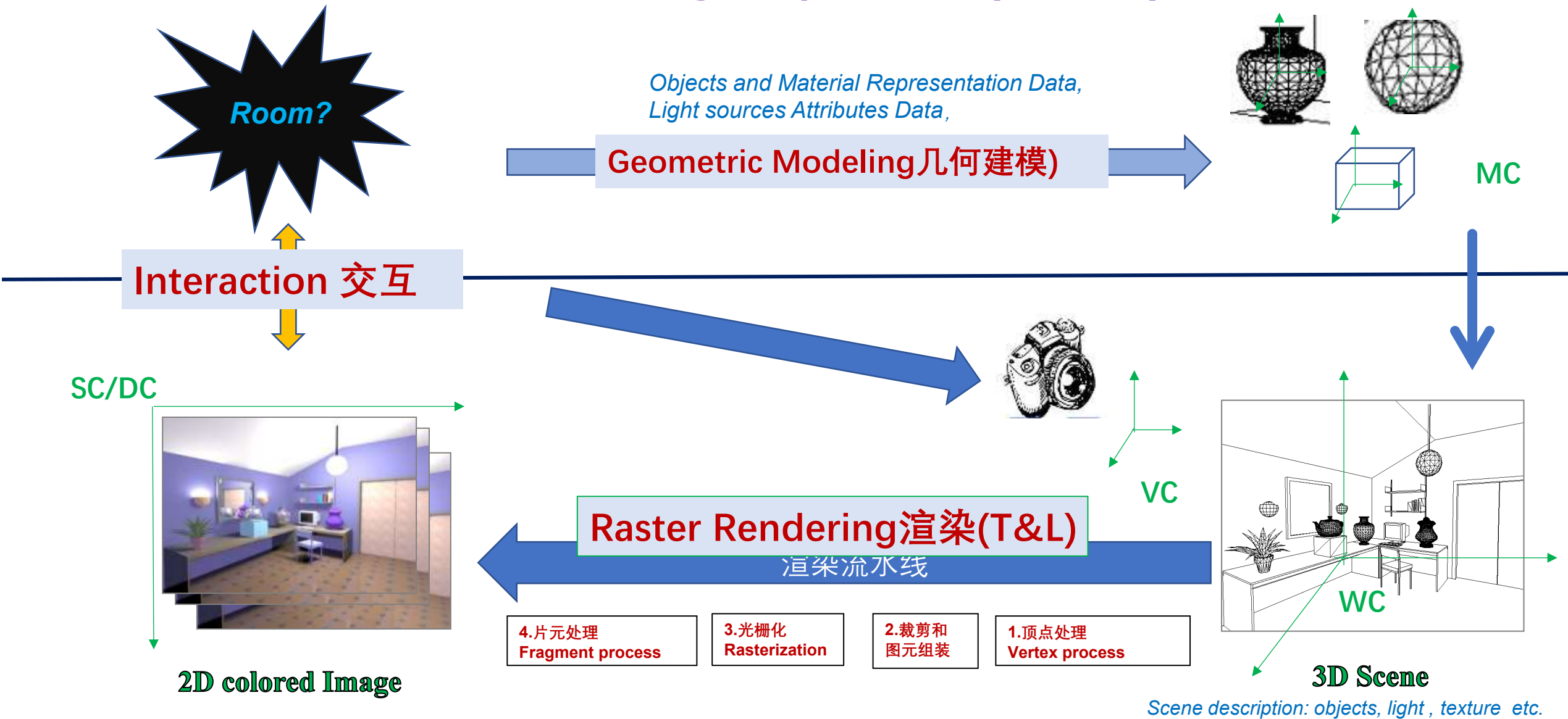


- 实际计算机实现时采用这种模型，并且通过技术实现“景深”效果。

Computer Graphics System Outlines

- Input devices
- Output device
- Process device*
- Image Formation Principle –成像原理
- **Rendering Pipeline –渲染管线***
- The Programmer's Interface-应用编程接口API

Rendering Pipeline(cont.)

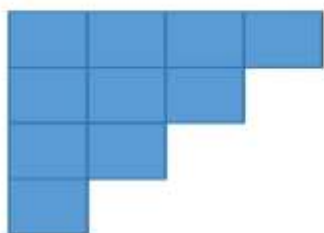


Rendering Pipeline(cont.)

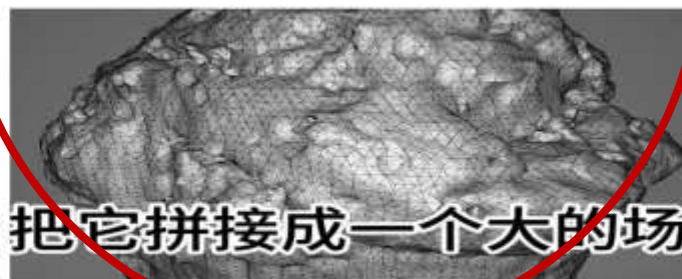
➤ Step1: Geometry 几何造型

- Triangular Mesh 三角形网格 (早期普遍采用的物体表面表示法)

方块组成的世界:

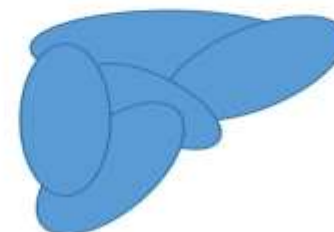


三角面组成的世界:



把它拼接成一个大的场景

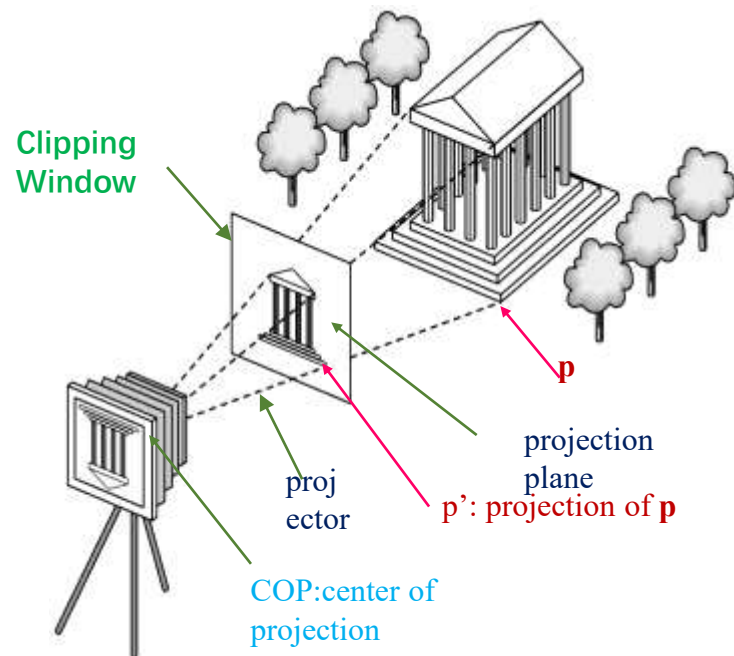
椭圆组成的世界:



Rendering Pipeline(cont.)

➤ Step2: Rendering (渲染)

➤ 给定场景物体，虚拟相机，照明等条件情况下，采用一些步骤生成2D图像的过程



渲染管线的主要功能：决定在给定虚拟相机、三维物体、光源、照明模式，以及纹理等诸多条件的情况下生成或绘制一幅二维图像的过程。

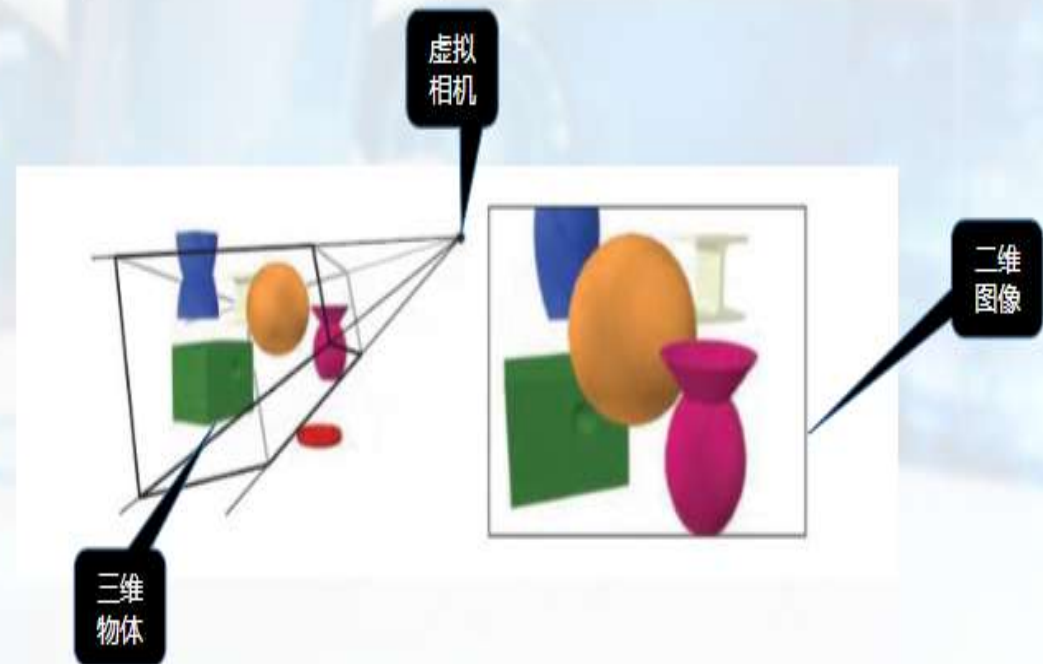
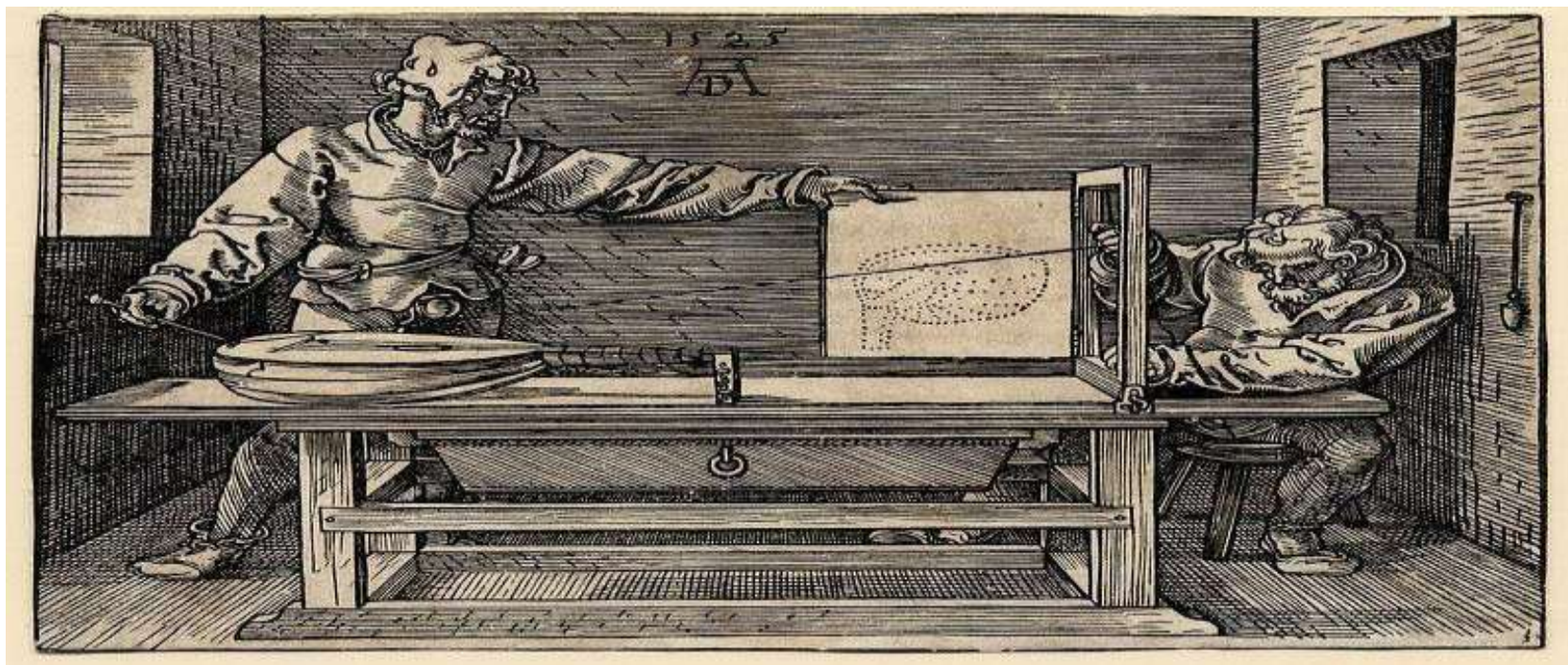


Image Formation Principle(cont.)

Step2: Rendering (渲染) ~续

渲染思路1: 正向采样 → 光栅化渲染算法 *shading*

- 从物体表面采样点出发，细线(投影线) 连接该采样点和墙上点 (投影中心点) 的直线交于合页 (投影面) 处画投影点颜色 (其它点靠插值填充)。



Ray Tracing vs. Rasterization(cont.)

Step2: Rendering (渲染) ~续

渲染思路2： 逆向采样→光线跟踪渲染 *ray tracing*

从投影中心点（眼睛）出发，视线穿过投影面方格（像素），交于物体表面，再将“看到”的内容绘制到对应图纸方格（像素）中。



Rendering Pipeline(cont.)

■ why Rendering Pipeline(为什么采用“渲染管线”)

➤ 图形处理采用管线 (Pipeline) 设计是计算机图形学领域一个非常核心且成功的设计范式。

➤ 管线设计能够高效、有序地处理“海量且步骤固定”的计算任务。(流水化且并行)

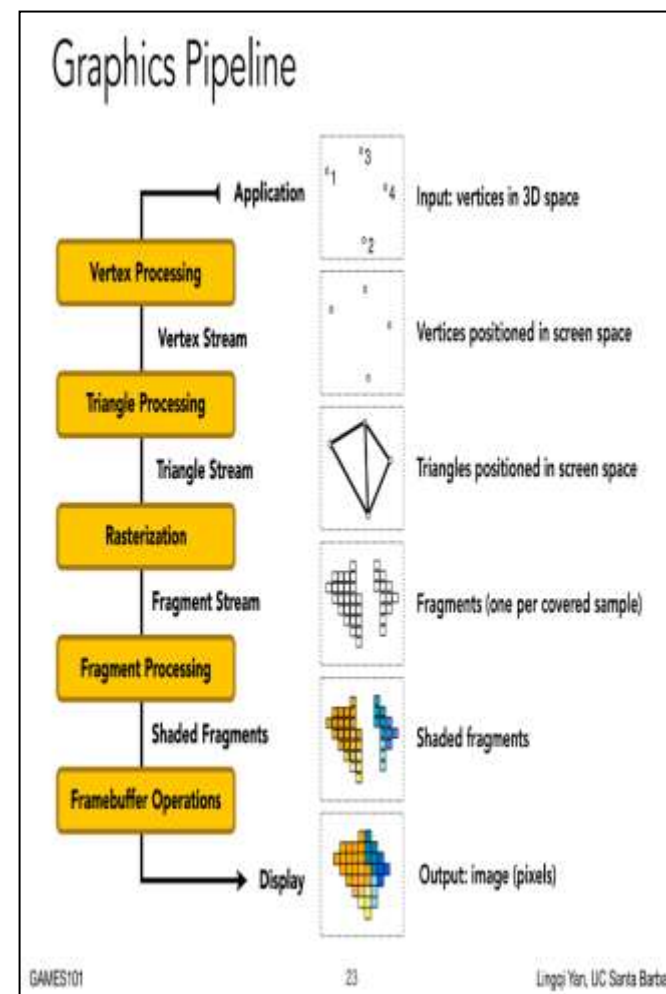
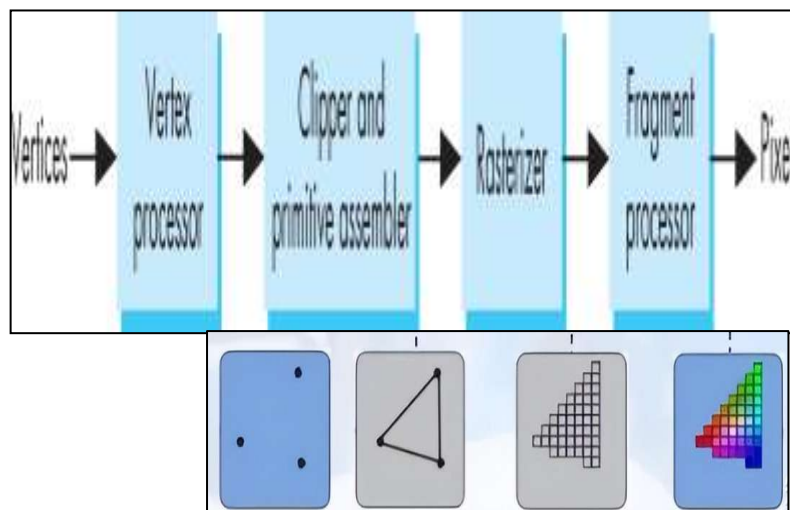
■ 把图形渲染想象成一个工厂的流水线。要生产100万辆汽车（即渲染数百万个三角形），最有效率的方式不是让一个工人从头到尾组装完一辆再组装下一辆，而是将组装过程分解为多个标准化的步骤（如安装底盘、装引擎、装车轮、喷漆），每个步骤由专门的工位（处理单元）负责。汽车（图形数据）在流水线上依次经过这些工位，最终成为成品。



Rendering Pipeline(cont.)

• Pipeline Steps (绘制管线的GPU主要操作步骤)

1. vertex process 顶点处理
2. Clipper and Primitive assembling 裁剪和图元组装
3. Rasterization 光栅化 (从图元到片元)
4. fragment processing 片元处理

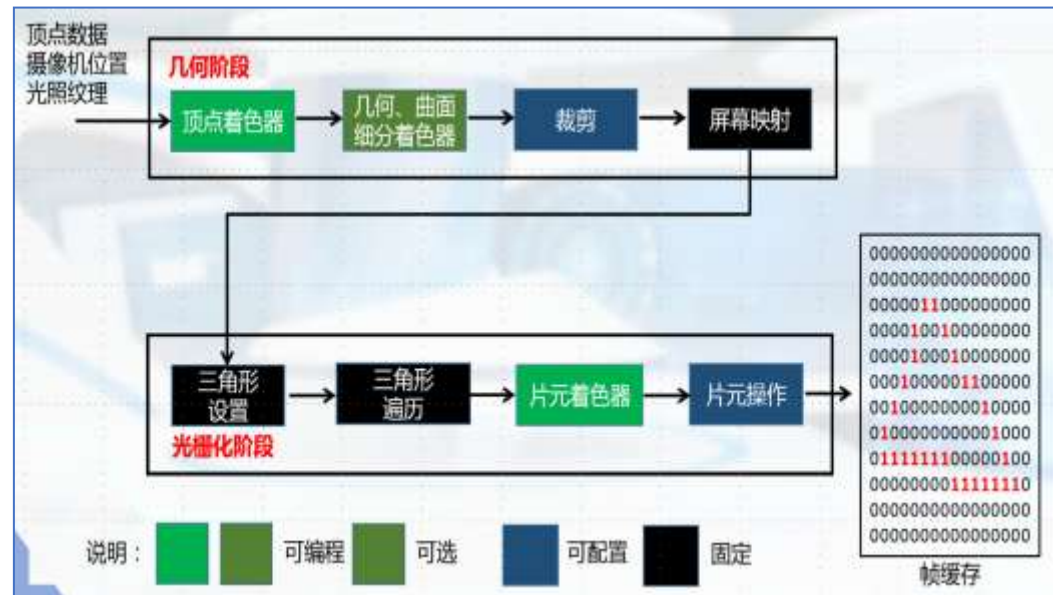
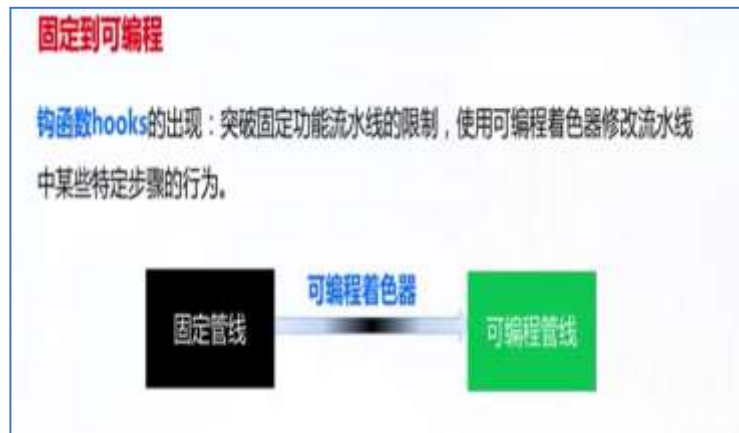


Rendering Pipeline(cont.)

➤ Programmable Rendering Pipeline(可编程渲染管线)

- The Fix Functions Pipeline(固定管线) : limited Functions 功能有限
 - GPU上提供的功能模块是“固定”，程序员只能通过接口API调用使用，但不能修改模块。
- The Programmable Pipeline(可编程): Extend Functions 功能扩展
 - GPU上的功能模块可以被程序员“编程”替换掉，可灵活扩展图形渲染流水线的功能。
 - 如“顶点着色器”，“片元着色器”，就是可由程序员编写shader程序替换的模块，

➤ GPU渲染管线上，有些模块是“可编程的”，有些是“固定的”



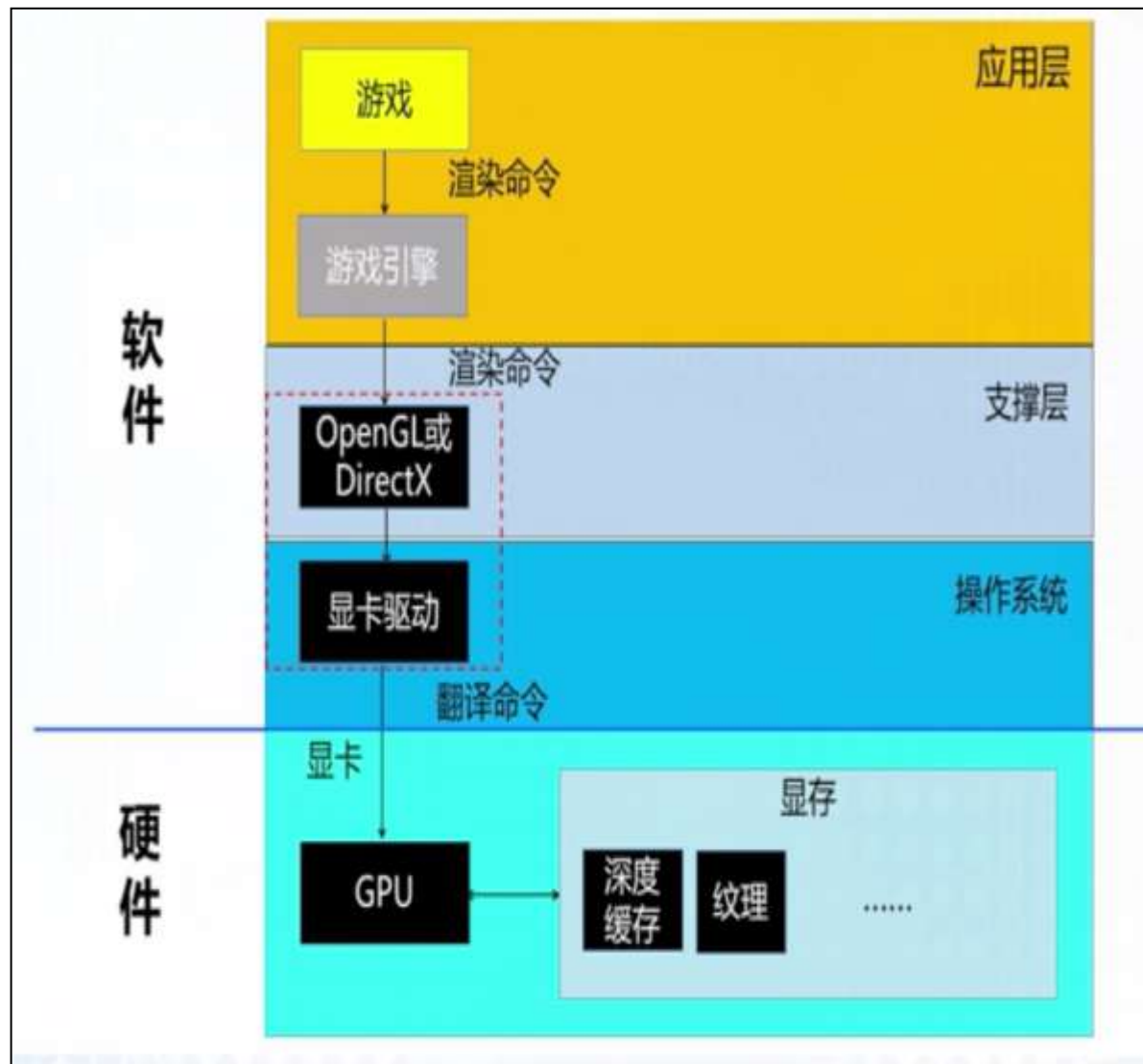
Computer Graphics System Outlines

- Input devices
 - Output device
 - Process device*
-
- Image Formation Principle –成像原理
 - Rendering Pipeline –渲染管线实现*
 - **The Programmer's Interface-应用编程接口API**

Application Programming Interface(API)

- 图形系统的软件层次
 - 应用层(游戏引擎)
 - 支撑层(图形库GL)
 - 操作系统 (显卡驱动)

从一个游戏看图形软件的构成：



Application Programming Interface (cont.)

➤ 图形系统的支撑层(图形库Graphics Library)的标准化:

- 标准化可以使图形库的编写不依赖于显卡硬件，所编写的程序具有可移植性



Application Programming Interface (cont.)

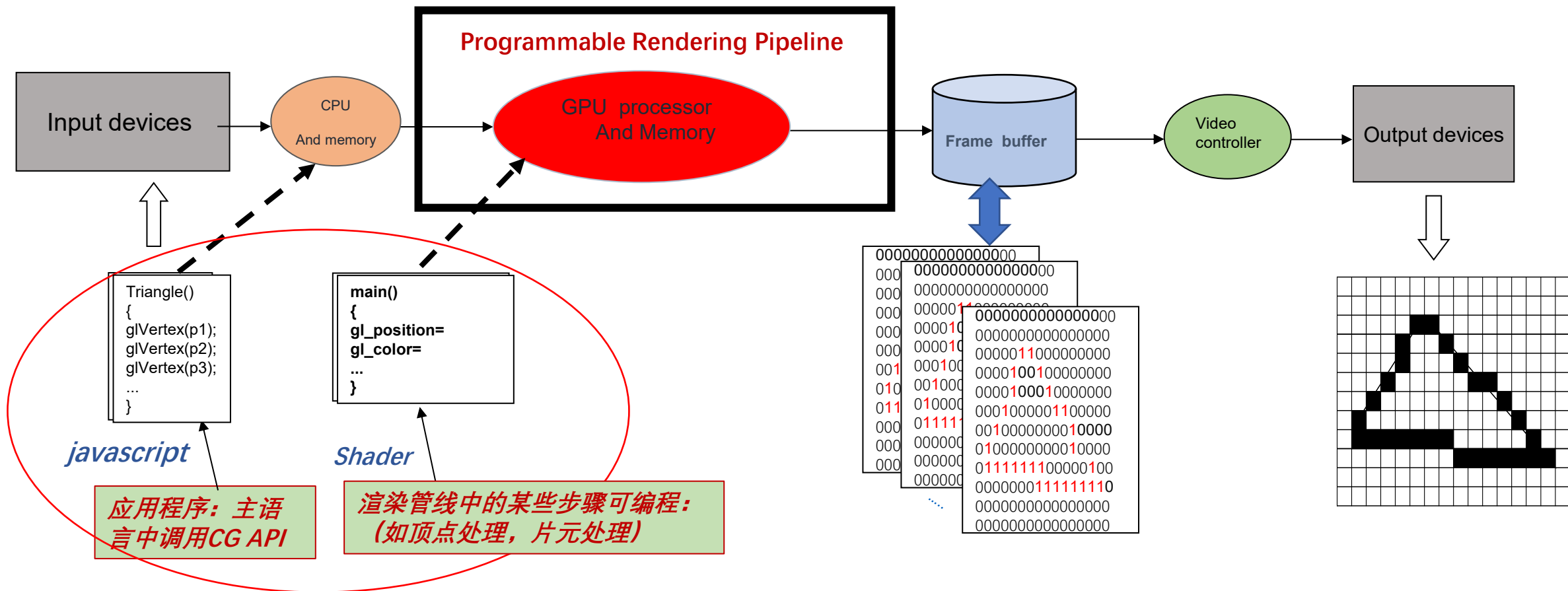
➤ 图形标准化库:

- Khronos标准库: 如OpenGL/webGL; Vulkan/webGPU; ref: <https://www.khronos.org/>



Application Programming Interface (cont.)

- CG application Programming



Summary

➤ 图形系统的硬件组成

- 输入设备

- 键盘，鼠标，绘图仪等

- 输出设备

- 光栅raster, 光点，物理分辨率，刷新频率refresh frequency，纵横比

- 处理及存储设备（显卡：显示子系统）

- 视频处理器controller：将帧缓存中的颜色编码用来控制显示器进行显示的模数转换功能部件
- 帧缓存framebuffer：存放一帧画面的所有像素的颜色编码的特定存储区
 - 像素pixel，分辨率resolution，帧frame, 帧缓存framebuffer
 - 颜色模型，像素深度/像素精度，位面Bit Plane，查色表CLT
- 显示处理器GPU：将程序中输入的参数转换为帧缓存中像素的颜色编码。

Summary(cont.)

➤图形系统的软件部分*

- Image Formation Principle –成像原理
 - 人眼成像，小孔成像，相机成像，虚拟相机模型
- Rendering Pipeline –渲染管线实现*
 - 固定管线，可编程渲染管线
- The Programmer's Interface-程序员编程接口
 - OpenGL/WebGL ， Vulkan/WebGPU